

សាកលវិទ្យាល័យ សៅស៍អ៊ីសថ៍អេយស្យៀ

UNIVERSITY OF SOUTH-EAST ASIA

Assignment Group I

Cloud Infrastructure & DevOps GitHub • Jenkins • Terraform • Docker • Docker Swarm • AWS EC2 • Nginx

Course : Advanced Cloud Technology

Lecturer : Mr. Krol Noeurn

Member Group : SARAN CHAMROEUN

SOM LIHENG

NGY LYTENG

ឆ្នាំសិក្សា ២០២៦-២០២៧

១. សេចក្តីផ្តើម

ក្នុងយុគសម័យឌីជីថលបច្ចុប្បន្ន បច្ចេកវិទ្យា Cloud Computing បានដើរតួនាទីយ៉ាងសំខាន់ក្នុងការអភិវឌ្ឍ ការដាក់ឱ្យដំណើរការ និងការគ្រប់គ្រងកម្មវិធីឱ្យមានប្រសិទ្ធភាព ភាពបត់បែន និងភាពងាយស្រួលក្នុងការពង្រីកប្រព័ន្ធ។ ទន្ទឹមនឹងនេះ ការអនុវត្តគោលការណ៍ DevOps និងការប្រើប្រាស់ឧបករណ៍ស្វ័យប្រវត្តិកម្ម បានជួយកាត់បន្ថយដំណើរការដែលត្រូវអនុវត្តដោយដៃ កាត់បន្ថយកំហុសក្នុងការកំណត់រចនាសម្ព័ន្ធ និងបង្កើនល្បឿននៃការអភិវឌ្ឍ និងការដាក់ឱ្យដំណើរការកម្មវិធី។

របាយការណ៍នេះផ្តោតលើការសិក្សា និងការអនុវត្តគម្រោង **Cloud Infrastructure & DevOps Automation** ដែលមានគោលបំណងរួមបញ្ចូលបច្ចេកវិទ្យា និងឧបករណ៍សំខាន់ៗ ដើម្បីបង្កើតហេដ្ឋារចនាសម្ព័ន្ធ Cloud និងរៀបចំដំណើរការដាក់ឱ្យដំណើរការកម្មវិធីដោយស្វ័យប្រវត្តិ។ ក្នុងគម្រោងនេះ Git និង GitHub ត្រូវបានប្រើប្រាស់សម្រាប់គ្រប់គ្រងប្រភពកូដ និងតាមដានការផ្លាស់ប្តូរ ខណៈ Jenkins ត្រូវបានប្រើសម្រាប់គ្រប់គ្រងដំណើរការ Continuous Integration និង Continuous Delivery/Deployment (CI/CD)។ Terraform ត្រូវបានប្រើប្រាស់ដើម្បីកំណត់ និងបង្កើតហេដ្ឋារចនាសម្ព័ន្ធតាមគោលការណ៍ Infrastructure as Code (IaC)។ ចំណែក Docker ត្រូវបានប្រើសម្រាប់វេចខ្ចប់កម្មវិធី និង Dependencies ជា Container ហើយ Docker Swarm ត្រូវបានប្រើសម្រាប់គ្រប់គ្រង និងដាក់ឱ្យដំណើរការ Containers នៅលើម៉ាស៊ីនមេច្រើន។

ហេដ្ឋារចនាសម្ព័ន្ធរបស់គម្រោងត្រូវបានរៀបចំនៅលើ Amazon Web Services (AWS) ដោយប្រើប្រាស់សេវាកម្ម Amazon Elastic Compute Cloud (EC2) សម្រាប់ផ្តល់ម៉ាស៊ីនមេនិម្មិត (Virtual Servers)។ លើសពីនេះ Nginx ត្រូវបានកំណត់ឱ្យដើរតួនាទីជា Load Balancer និង Reverse Proxy ដើម្បីទទួលសំណើពីអ្នកប្រើប្រាស់ និងបញ្ជូនសំណើទាំងនោះទៅកាន់សេវាកម្មខាងក្រោយ (Backend Services) ឱ្យបានសមស្រប។

គោលបំណងសំខាន់នៃការសិក្សានេះ គឺដើម្បីស្វែងយល់ពីរបៀបភ្ជាប់ និងប្រើប្រាស់បច្ចេកវិទ្យាទាំងនេះរួមគ្នា ក្នុងការបង្កើតលំហូរការងារដែលមានលក្ខណៈស្វ័យប្រវត្តិ និងអាចអនុវត្តឡើងវិញបាន។ ដំណើរការនេះគ្របដណ្តប់លើការគ្រប់គ្រងប្រភពកូដ ការបង្កើតកម្មវិធី (Build) ការធ្វើតេស្ត (Test) ការបង្កើតហេដ្ឋារចនាសម្ព័ន្ធ (Infrastructure Provisioning) ការដាក់ឱ្យដំណើរការ (Deployment) និងការផ្ទៀងផ្ទាត់លទ្ធផល (Verification)។ ក្រៅពីនេះ គម្រោងក៏រួមបញ្ចូលការគ្រប់គ្រងសុវត្ថិភាព ការធ្វើបច្ចុប្បន្នភាពកម្មវិធី ការត្រឡប់ទៅកំណែមុន (Rollback) និងការជូនដំណឹងអំពីស្ថានភាព CI/CD Pipeline តាមរយៈ Telegram ផងដែរ។

តាមរយៈការអនុវត្តគម្រោងនេះ ក្រុមសិក្សាអាចទទួលបានចំណេះដឹង និងបទពិសោធន៍ជាក់ស្តែងអំពីការរៀបចំហេដ្ឋារចនាសម្ព័ន្ធ Cloud ការគ្រប់គ្រង Containers និងការប្រើប្រាស់ឧបករណ៍ DevOps ដើម្បីគាំទ្រដល់ការអភិវឌ្ឍ និងការដាក់ឱ្យដំណើរការកម្មវិធី។ លទ្ធផលនៃការសិក្សានឹងត្រូវបានបង្ហាញតាមរយៈការកំណត់រចនាសម្ព័ន្ធប្រព័ន្ធ ដំណាក់កាលអនុវត្ត និងកស្តុតាងនៃការធ្វើតេស្ត ដើម្បីវាយតម្លៃប្រសិទ្ធភាព និងកំណត់ចំណុចដែលអាចកែលម្អបន្ថែមនាពេលអនាគត។

២. ល័ង្កាមញ្ហា

ក្នុងការអភិវឌ្ឍ និងដាក់ឱ្យដំណើរការកម្មវិធីលើប្រព័ន្ធ Cloud ការគ្រប់គ្រងប្រភពកូដ ការកំណត់បរិស្ថានដំណើរការ ហេដ្ឋារចនាសម្ព័ន្ធ និងការតភ្ជាប់បណ្តាញ គឺជាកត្តាសំខាន់ដែលត្រូវការការសម្របសម្រួលឱ្យបានត្រឹមត្រូវ។ ជាទូទៅ ប្រសិនបើការបង្កើតម៉ាស៊ីនមេ (Server) ការដំឡើង Dependencies ការកំណត់រចនាសម្ព័ន្ធប្រព័ន្ធ និងការដាក់ឱ្យដំណើរការកំណែកម្មវិធីថ្មី ត្រូវបានអនុវត្តដោយដៃ នោះអាចបណ្តាលឱ្យចំណាយពេលវេលាច្រើន បង្កើនភាពស្មុគស្មាញ និងងាយកើតមានកំហុសក្នុងដំណើរការអនុវត្ត។ ជាពិសេស ភាពខុសគ្នារវាងបរិស្ថានអភិវឌ្ឍកម្មវិធីនៅលើម៉ាស៊ីនផ្ទាល់ (Local Environment) និងបរិស្ថាន Cloud អាចធ្វើឱ្យកម្មវិធីដែលដំណើរការបានល្អនៅលើ Local ជួបប្រទះបញ្ហានៅពេលដាក់ឱ្យដំណើរការលើ Cloud។

លើសពីនេះ ការគ្រប់គ្រងកម្មវិធីដែលដំណើរការលើម៉ាស៊ីនមេច្រើន (Multiple Nodes) ក៏ជាបញ្ហាប្រឈមមួយផងដែរ។ ការបែងចែកសេវាកម្ម ការបញ្ជូនសំណើពីអ្នកប្រើប្រាស់ទៅកាន់ Backend Services និងការគ្រប់គ្រងចំនួន Replicas តម្រូវឱ្យមានការកំណត់រចនាសម្ព័ន្ធ និងការត្រួតពិនិត្យស្ថានភាពប្រព័ន្ធឱ្យបានច្បាស់លាស់។ ទន្ទឹមនឹងនេះ ការធ្វើបច្ចុប្បន្នភាពកម្មវិធី (Rolling Update) ឬការត្រឡប់ទៅកំណែមុន (Rollback) ប្រសិនបើមិនមានដំណើរការត្រួតពិនិត្យ និងផ្ទៀងផ្ទាត់ត្រឹមត្រូវ អាចបង្កឱ្យមានការរំខានដល់សេវាកម្ម និងធ្វើឱ្យពិបាកវាយតម្លៃថាប្រព័ន្ធនៅតែអាចដំណើរការបានប្រក្រតីឬអត់។

បញ្ហាសុវត្ថិភាពក៏ជាចំណុចសំខាន់ដែលត្រូវយកចិត្តទុកដាក់ផងដែរ។ ការរក្សាទុកព័ត៌មានសម្ងាត់ដូចជា Credentials, Access Keys និង Secrets នៅក្នុងប្រភពកូដដោយផ្ទាល់ អាចបង្កើនហានិភ័យនៃការលេចធ្លាយព័ត៌មាន។ ដូចគ្នានេះដែរ ការកំណត់សិទ្ធិចូលប្រើបណ្តាញលើសពីតម្រូវការដាក់ស្តែង អាចបង្កើតចន្លោះប្រហោងផ្នែកសុវត្ថិភាព និងប៉ះពាល់ដល់ភាពជឿជាក់នៃប្រព័ន្ធ។ ដូច្នេះ ការរៀបចំដំណើរការដែលមានស្វ័យប្រវត្តិកម្ម មានសុវត្ថិភាព និងអាចអនុវត្តឡើងវិញបាន គឺជាតម្រូវការចាំបាច់សម្រាប់ការអភិវឌ្ឍ និងគ្រប់គ្រងកម្មវិធីលើ Cloud។

ដើម្បីដោះស្រាយបញ្ហាប្រឈមទាំងនេះ គម្រោង **Cloud Infrastructure & DevOps Automation** ត្រូវបានរៀបចំឡើងដោយផ្តោតលើការរួមបញ្ចូលបច្ចេកវិទ្យា GitHub, Jenkins, Terraform, Docker, Docker Swarm, AWS EC2 និង Nginx ក្នុងដំណើរការការងារតែមួយ។ ការរួមបញ្ចូលនេះមានគោលបំណងកាត់បន្ថយការកំណត់ដោយដៃ បង្កើនប្រសិទ្ធភាពនៃការគ្រប់គ្រងហេដ្ឋារចនាសម្ព័ន្ធ និងរៀបចំលំហូរការងារ CI/CD ដើម្បីគាំទ្រដល់ការបង្កើត ការធ្វើតេស្ត និងការដាក់ឱ្យដំណើរការកម្មវិធីដោយស្វ័យប្រវត្តិ។

ដោយផ្អែកលើបញ្ហាខាងលើ សំណួរសំខាន់នៃការសិក្សានេះគឺ៖ តើការរួមបញ្ចូលបច្ចេកវិទ្យា GitHub, Jenkins, Terraform, Docker, Docker Swarm, AWS EC2 និង Nginx អាចជួយបង្កើតលំហូរការងារដែលមានស្វ័យប្រវត្តិកម្ម សុវត្ថិភាព និងប្រសិទ្ធភាព ចាប់ពីការគ្រប់គ្រងប្រភពកូដ រហូតដល់ការដាក់ឱ្យដំណើរការ និងការផ្ទៀងផ្ទាត់លទ្ធផលកម្មវិធីនៅលើ Cloud បានយ៉ាងដូចម្តេច?

៣. វត្ថុបំណងនៃគម្រោង

គម្រោង Cloud Infrastructure & DevOps Automation មានគោលបំណងសំខាន់ក្នុងការសិក្សា និងអនុវត្តបច្ចេកវិទ្យា Cloud Computing រួមជាមួយគោលការណ៍ DevOps ដើម្បីរៀបចំហេដ្ឋារចនាសម្ព័ន្ធ Cloud និងបង្កើតដំណើរការអភិវឌ្ឍ និងដាក់ឱ្យដំណើរការកម្មវិធីដោយស្វ័យប្រវត្តិ។ គម្រោងនេះផ្តោតលើការរួមបញ្ចូលឧបករណ៍ និងបច្ចេកវិទ្យាផ្សេងៗក្នុងលំហូរការងារតែមួយ ចាប់ពីការគ្រប់គ្រងប្រភពកូដ ការបង្កើតកម្មវិធី ការធ្វើតេស្ត ការរៀបចំហេដ្ឋារចនាសម្ព័ន្ធ រហូតដល់ការដាក់ឱ្យដំណើរការ និងការឆ្លើងផ្ទាត់លទ្ធផលនៅលើ Cloud។
ដើម្បីសម្រេចបាននូវគោលបំណងខាងលើ គម្រោងនេះមានវត្ថុបំណងជាក់លាក់ដូចខាងក្រោម៖

១. ការគ្រប់គ្រងប្រភពកូដ (Source Code Management)៖ សិក្សា និងប្រើប្រាស់ Git និង GitHub សម្រាប់រក្សាទុក និងគ្រប់គ្រងប្រភពកូដ ឯកសារកំណត់រចនាសម្ព័ន្ធ និងប្រវត្តិនៃការផ្លាស់ប្តូរកម្មវិធី ដើម្បីធានាឱ្យការអភិវឌ្ឍមានរបៀបរៀបរយ ងាយស្រួលតាមដាន និងអាចសហការជាក្រុមបានប្រកបដោយប្រសិទ្ធភាព។

២. ការវេចខ្ចប់ និងសាកល្បងកម្មវិធី (Containerization and Local Testing)៖ ប្រើប្រាស់ Docker ដើម្បីវេចខ្ចប់កម្មវិធី និង Dependencies ទៅជា Docker Images ហើយបញ្ជូនទៅកាន់ Docker Hub។ បន្ទាប់មក ប្រើប្រាស់ Docker Compose សម្រាប់ដាក់ឱ្យដំណើរការ និងសាកល្បងការតភ្ជាប់រវាងសេវាកម្មផ្សេងៗនៅលើ Local Environment មុនពេលដាក់ឱ្យដំណើរការនៅលើ Cloud។

៣. ការរៀបចំហេដ្ឋារចនាសម្ព័ន្ធ Cloud (Cloud Infrastructure Provisioning)៖ សិក្សា និងប្រើប្រាស់ Terraform ដើម្បីកំណត់ និងបង្កើតហេដ្ឋារចនាសម្ព័ន្ធនៅលើ AWS ដូចជា EC2 Instances, Networking និង Security Groups តាមគោលការណ៍ Infrastructure as Code (IaC)។ ការអនុវត្តនេះមានគោលបំណងកាត់បន្ថយការកំណត់ដោយដៃ និងធ្វើឱ្យការបង្កើតហេដ្ឋារចនាសម្ព័ន្ធអាចអនុវត្តឡើងវិញបាន។

៤. ការគ្រប់គ្រង Container លើ Cloud (Container Orchestration)៖ រៀបចំ Docker Swarm Cluster ដែលមាន Manager Node និង Worker Nodes ដើម្បីគ្រប់គ្រងការដាក់ឱ្យដំណើរការកម្មវិធីតាមរយៈ Docker Stack ព្រមទាំងគ្រប់គ្រង Services និង Replicas នៅលើម៉ាស៊ីនមេច្រើនឱ្យមានភាពងាយស្រួល និងមានប្រសិទ្ធភាព។

៥. ការគ្រប់គ្រងចរាចរណ៍បណ្តាញ (Load Balancing and Reverse Proxy)៖ កំណត់ Nginx ឱ្យដើរតួនាទីជា Load Balancer និង Reverse Proxy សម្រាប់ទទួលសំណើពីអ្នកប្រើប្រាស់ និងបញ្ជូនសំណើទៅកាន់ Backend Services។ បន្ទាប់មក ធ្វើការសោកល្បងការចូលប្រើកម្មវិធីតាមរយៈ HTTP ដើម្បីឆ្លើងផ្ទាត់ការតភ្ជាប់ និងការឆ្លើយតបរបស់ប្រព័ន្ធ។

៦. ការបង្កើតដំណើរការស្វ័យប្រវត្តិ CI/CD (CI/CD Automation)៖ ភ្ជាប់ GitHub ជាមួយ Jenkins ដើម្បីរៀបចំ CI/CD Pipeline ដែលរួមមានដំណាក់កាល Checkout, Build, Test, Push Image, Deploy និង Verify។ ការអនុវត្តនេះមានគោលបំណងកាត់បន្ថយការងារដោយដៃ និងបង្កើនប្រសិទ្ធភាពក្នុងការដាក់ឱ្យដំណើរការកម្មវិធី។

៧. ការគ្រប់គ្រងកំណែកម្មវិធី (Application Version Management)៖ សិក្សា និងអនុវត្តការធ្វើ Rolling Update ដើម្បីដាក់ឱ្យដំណើរការកំណែកម្មវិធីថ្មី និងការធ្វើ Rollback ដើម្បីត្រឡប់ទៅកំណែមុននៅពេលចាំបាច់។ ទន្ទឹមនឹងនេះ ត្រូវត្រួតពិនិត្យស្ថានភាព Services និង Replicas ដើម្បីឆ្លើងផ្ទាត់ដំណើរការរបស់ប្រព័ន្ធក្រោយធ្វើបច្ចុប្បន្នភាព។

៨. ការគ្រប់គ្រងសុវត្ថិភាពប្រព័ន្ធ (Security and Secret Management)៖ អនុវត្តការគ្រប់គ្រង Credentials, Access Keys និង Secrets ឱ្យមានសុវត្ថិភាព ដោយជៀសវាងការរក្សាទុកព័ត៌មានសម្ងាត់នៅក្នុងប្រភពកូដដោយផ្ទាល់។ លើសពីនេះ កំណត់សិទ្ធិចូលប្រើបណ្តាញ និង Security Groups ឱ្យសមស្របតាមតម្រូវការរបស់គម្រោង។

៩. ការជូនដំណឹងអំពីស្ថានភាពប្រព័ន្ធ (Pipeline Notification)៖ រៀបចំការជូនដំណឹងតាមរយៈ Telegram Bot ដែលភ្ជាប់ជាមួយ Jenkins ដើម្បីផ្ញើព័ត៌មានអំពីស្ថានភាពនៃ CI/CD Pipeline ដូចជា Build, Deploy និង Rollback ព្រមទាំងលទ្ធផលជោគជ័យ ឬបរាជ័យ ដើម្បីឱ្យសមាជិកក្រុមអាចតាមដានដំណើរការបានទាន់ពេលវេលា។

១០. ការផ្ទៀងផ្ទាត់ និងវាយតម្លៃលទ្ធផល (Verification and Evaluation)៖ ធ្វើការត្រួតពិនិត្យ កត់ត្រា និងវាយតម្លៃលទ្ធផលនៃការអនុវត្តគម្រោង ដោយផ្អែកលើកស្តុតាងជាក់ស្តែង ដូចជា Jenkins Build Status, HTTP Response, Docker Services, Replicas និងលទ្ធផលនៃការដាក់ឱ្យដំណើរការកម្មវិធីនៅលើ AWS EC2។

តាមរយៈការសម្រេចបាននូវវត្ថុបំណងទាំងនេះ គម្រោងរំពឹងថានឹងអាចបង្ហាញពីរបៀបរួមបញ្ចូលបច្ចេកវិទ្យា Cloud Computing និង DevOps ក្នុងការរៀបចំប្រព័ន្ធដែលមានលំហូរការងារស្វ័យប្រវត្តិ មានសុវត្ថិភាព និងអាចគ្រប់គ្រងបានយ៉ាងមានប្រសិទ្ធភាព។ ជាមួយគ្នានេះ ការអនុវត្តគម្រោងក៏ជួយបង្កើនចំណេះដឹង និងបទពិសោធន៍ជាក់ស្តែងក្នុងការប្រើប្រាស់បច្ចេកវិទ្យាទាំងនេះ សម្រាប់ការអភិវឌ្ឍ និងដាក់ឱ្យដំណើរការកម្មវិធីនៅលើ Cloud ផងដែរ។

៤. បច្ចេកទេសដែលប្រើប្រាស់ក្នុងគម្រោង

ក្នុងការអនុវត្តគម្រោង **Cloud Infrastructure & DevOps Automation** បច្ចេកវិទ្យា និងឧបករណ៍ជាច្រើនត្រូវបានប្រើប្រាស់រួមគ្នា ដើម្បីរៀបចំហេដ្ឋារចនាសម្ព័ន្ធ Cloud និងបង្កើតលំហូរការងារអភិវឌ្ឍ និងដាក់ឱ្យដំណើរការកម្មវិធីដោយស្វ័យប្រវត្តិ។ បច្ចេកវិទ្យានីមួយៗមានតួនាទីផ្សេងគ្នា ប៉ុន្តែអាចធ្វើការរួមគ្នាបាន ចាប់ពីការគ្រប់គ្រងប្រភពកូដ ការរៀចខ្ចប់កម្មវិធី ការបង្កើតហេដ្ឋារចនាសម្ព័ន្ធ រហូតដល់ការគ្រប់គ្រងសេវាកម្ម និងការត្រួតពិនិត្យលទ្ធផល។ បច្ចេកវិទ្យាសំខាន់ៗដែលប្រើប្រាស់ក្នុងគម្រោងមានដូចខាងក្រោម៖

៤.១. Docker

Docker គឺជាបច្ចេកវិទ្យា Containerization ដែលអនុញ្ញាតឱ្យអ្នកអភិវឌ្ឍកម្មវិធីវេចខ្ចប់ Application រួមជាមួយ Libraries, Dependencies និងការកំណត់ចាំបាច់ផ្សេងៗ ទៅជា Docker Image។ Image នេះអាចត្រូវបានប្រើប្រាស់ដើម្បីបង្កើត និងដាក់ឱ្យដំណើរការ Container នៅលើបរិស្ថានផ្សេងៗបានយ៉ាងងាយស្រួល។

ការប្រើប្រាស់ Docker ជួយកាត់បន្ថយបញ្ហាភាពខុសគ្នារវាងបរិស្ថានអភិវឌ្ឍកម្មវិធី (Development Environment) និងបរិស្ថានដាក់ឱ្យដំណើរការ (Production Environment) ដោយធ្វើឱ្យកម្មវិធីមានបរិស្ថានដំណើរការស្របគ្នា។ លើសពីនេះ Docker ក៏ជួយសម្រួលដល់ការចែកចាយកម្មវិធី ការគ្រប់គ្រងកំណែ និងការដាក់ឱ្យដំណើរការកម្មវិធីលើ Cloud ផងដែរ។

ក្នុងគម្រោងនេះ Docker ត្រូវបានប្រើប្រាស់សម្រាប់បង្កើត Docker Images របស់កម្មវិធី បញ្ជូន Images ទៅកាន់ Docker Hub និងសាកល្បងការដំណើរការរបស់សេវាកម្មនៅលើ Local Environment មុនពេលដាក់ឱ្យដំណើរការលើ AWS EC2 ។

សមាសភាគសំខាន់ៗរបស់ Docker រួមមាន៖

- Docker Image៖ ជាគំរូដែលផ្ទុកកម្មវិធី និងសមាសភាគចាំបាច់ សម្រាប់បង្កើត Docker Container។
- Docker Container៖ ជាបរិស្ថានដំណើរការដែលត្រូវបានបង្កើតចេញពី Docker Image។
- Dockerfile៖ ជាឯកសារដែលផ្ទុកសំណុំ Instructions សម្រាប់កំណត់ដំណាក់កាលបង្កើត Docker Image។
- Docker Compose៖ ជាឧបករណ៍ដែលប្រើ YAML កំណត់និងគ្រប់គ្រងកម្មវិធីមាន Containers ច្រើន។
- Container Registry៖ ជាទីតាំងសម្រាប់រក្សាទុក និងចែកចាយ Docker Images ដើម្បីឱ្យ Servers ឬ Docker Swarm Nodes អាចទាញយក Images ទៅប្រើប្រាស់បាន។



រូបភាពទី៤.១៖ Logo Docker

៤.២. Terraform

Terraform គឺជាឧបករណ៍សម្រាប់គ្រប់គ្រង និងបង្កើតហេដ្ឋារចនាសម្ព័ន្ធតាមគោលការណ៍ Infrastructure as Code (IaC)។ បច្ចេកវិទ្យានេះអនុញ្ញាតឱ្យអ្នកអភិវឌ្ឍកំណត់រចនាសម្ព័ន្ធធនធាន Cloud តាមរយៈ Configuration Files ជំនួសឱ្យការបង្កើត និងកំណត់ធនធានដោយដៃ។

ការប្រើប្រាស់ Terraform ជួយធ្វើឱ្យការរៀបចំហេដ្ឋារចនាសម្ព័ន្ធមានស្តង់ដារ អាចអនុវត្តឡើងវិញបាន និងងាយស្រួលក្នុងការគ្រប់គ្រងការផ្លាស់ប្តូរ។ អ្នកអភិវឌ្ឍអាចកំណត់ធនធានដែលត្រូវការជាមុន ពិនិត្យមើលការផ្លាស់ប្តូរ និងអនុវត្តការបង្កើត ឬកែប្រែហេដ្ឋារចនាសម្ព័ន្ធតាម Configuration ដែលបានរៀបចំ។

ក្នុងគម្រោងនេះ Terraform ត្រូវបានប្រើប្រាស់ដើម្បីបង្កើត និងគ្រប់គ្រងធនធាននៅលើ AWS ដូចជា Virtual Private Cloud (VPC), Subnets, Security Groups, EC2 Instances និង Key Pairs។ ការអនុវត្តនេះជួយកាត់បន្ថយការងារដោយដៃ និងធ្វើឱ្យការរៀបចំហេដ្ឋារចនាសម្ព័ន្ធមានភាពស្របគ្នា។

ពាក្យបញ្ជា និងឯកសារសំខាន់ៗរបស់ Terraform រួមមាន៖

- terraform init៖ សម្រាប់រៀបចំបរិស្ថានការងារដំបូង និងទាញយក Providers ដែលចាំបាច់។

- terraform plan៖ សម្រាប់ពិនិត្យមើលការផ្លាស់ប្តូរធនធានដែលនឹងត្រូវអនុវត្ត។
- terraform apply៖ សម្រាប់បង្កើត ឬកែប្រែហេដ្ឋារចនាសម្ព័ន្ធតាម Configuration Files។
- terraform destroy៖ សម្រាប់លុបធនធានដែលគ្រប់គ្រងដោយ Terraform នៅពេលលែងត្រូវការ។
- variables.tf៖ សម្រាប់កំណត់ Input Variables ដែលប្រើប្រាស់ក្នុង Configuration។
- outputs.tf៖ សម្រាប់កំណត់ព័ត៌មានលទ្ធផលសំខាន់ៗ ដូចជា Public IP Address របស់ EC2 Instances។



រូបភាពទី៤.២៖ Logo Terraform

៤.៣. Git និង GitHub

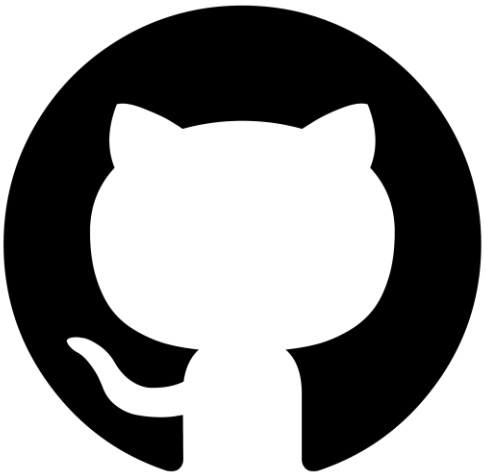
Git គឺជាប្រព័ន្ធគ្រប់គ្រងកំណែ (Version Control System) ដែលប្រើប្រាស់សម្រាប់តាមដាន និងរក្សាទុកប្រវត្តិនៃការផ្លាស់ប្តូរប្រភពកូដ។ ចំណែក GitHub គឺជាវេទិកាសម្រាប់រក្សាទុក Git Repositories លើអ៊ីនធឺណិត ដែលអនុញ្ញាតឱ្យអ្នកអភិវឌ្ឍធ្វើការសហការគ្នា គ្រប់គ្រងគម្រោង និងត្រួតពិនិត្យការផ្លាស់ប្តូរកូដបានយ៉ាងងាយស្រួល។

ការប្រើប្រាស់ Git និង GitHub ជួយឱ្យការអភិវឌ្ឍកម្មវិធីមានរបៀបរៀបរយ និងអាចតាមដានប្រវត្តិនៃការកែប្រែបានច្បាស់លាស់។ ជាពិសេស វាអនុញ្ញាតឱ្យសមាជិកក្រុមអភិវឌ្ឍមុខងារផ្សេងៗនៅលើ Branch ដាច់ដោយឡែក មុនពេលពិនិត្យ និងបញ្ចូលការផ្លាស់ប្តូរទៅក្នុង Main Branch។

ក្នុងគម្រោងនេះ GitHub ត្រូវបានប្រើប្រាស់ជាទីតាំងកណ្តាលសម្រាប់រក្សាទុក Source Code, Dockerfile, Jenkinsfile, Terraform Configuration និងឯកសារពាក់ព័ន្ធផ្សេងៗ។ លើសពីនេះ GitHub ក៏ត្រូវបានភ្ជាប់ជាមួយ Jenkins ដើម្បីគាំទ្រដល់ដំណើរការ CI/CD Pipeline ផងដែរ។

សមាសភាគសំខាន់ៗរួមមាន៖

- Repository៖ ជាទីតាំងសម្រាប់រក្សាទុក Source Code និងឯកសាររបស់គម្រោង។
- Commit៖ ជាការកត់ត្រាការផ្លាស់ប្តូរប្រភពកូដនៅក្នុងប្រវត្តិរបស់ Git។
- Branch៖ ជាផ្នែកដាច់ដោយឡែកសម្រាប់អភិវឌ្ឍមុខងារថ្មី ឬកែប្រែកំហុស ដោយមិនប៉ះពាល់ដល់ Main Branch ដោយផ្ទាល់។
- Pull Request៖ ជាដំណើរការស្នើសុំឱ្យពិនិត្យ និងបញ្ចូលការផ្លាស់ប្តូរពី Branch មួយទៅ Branch មួយទៀត។



រូបភាពទី៤.៣៖ Logo Git និង GitHub

៤.៤. Jenkins និង CI/CD

Jenkins គឺជាឧបករណ៍ Automation Server ដែលត្រូវបានប្រើប្រាស់សម្រាប់រៀបចំ និងគ្រប់គ្រងដំណើរការស្វ័យប្រវត្តិកម្មក្នុងការអភិវឌ្ឍ និងជាក់ឱ្យដំណើរការកម្មវិធី។ Jenkins មានតួនាទីសំខាន់ក្នុងការអនុវត្តគោលការណ៍ Continuous Integration (CI) និង Continuous Delivery/Deployment (CD) ដើម្បីធ្វើឱ្យដំណើរការ Build, Test និង Deploy កម្មវិធីអាចអនុវត្តបានយ៉ាងមានប្រសិទ្ធភាព។

Jenkins អាចប្រើប្រាស់ Jenkinsfile ដើម្បីកំណត់លំដាប់ដំណើរការរបស់ Pipeline ជា Code ដែលអាចរក្សាទុក និងគ្រប់គ្រងកំណែជាមួយ Source Code បាន។ លើសពីនេះ Jenkins អាចភ្ជាប់ជាមួយ GitHub តាមរយៈ Webhook ដើម្បីចាប់ផ្តើម Pipeline ដោយស្វ័យប្រវត្តិ នៅពេលមានការផ្លាស់ប្តូរប្រភពកូដ។

ក្នុងគម្រោងនេះ Jenkins ត្រូវបានប្រើប្រាស់សម្រាប់គ្រប់គ្រង CI/CD Pipeline ដែលមានដំណាក់កាលសំខាន់ៗដូចជា Checkout, Build, Test, Push Docker Image, Deploy ទៅ Docker Swarm និង Verify លទ្ធផល។ Jenkins ក៏ត្រូវបានប្រើប្រាស់សម្រាប់គ្រប់គ្រង Rolling Update, Rollback និងការជូនដំណឹងតាម Telegram ផងដែរ។

សមាសភាគសំខាន់ៗរួមមាន៖

- Continuous Integration (CI)៖ ជាដំណើរការបញ្ចូល និងធ្វើតេស្តការផ្លាស់ប្តូរប្រភពកូដជាបន្តបន្ទាប់។
- Continuous Delivery/Deployment (CD)៖ ជាដំណើរការរៀបចំ ឬជាក់ឱ្យដំណើរការកំណែកម្មវិធីដែលបានឆ្លងកាត់ការត្រួតពិនិត្យ និងធ្វើតេស្ត។
- Jenkinsfile៖ ជាឯកសារសម្រាប់កំណត់ Stages និង Instructions របស់ CI/CD Pipeline។
- Jenkins Credentials៖ ជាមុខងារសម្រាប់គ្រប់គ្រងព័ត៌មានសម្ងាត់ ដូចជា Access Tokens, Passwords និង SSH Keys ដោយមិនចាំបាច់បញ្ចូលព័ត៌មានទាំងនោះក្នុង Source Code ដោយផ្ទាល់។



រូបភាពទី៤.៤៖ Logo Jenkins

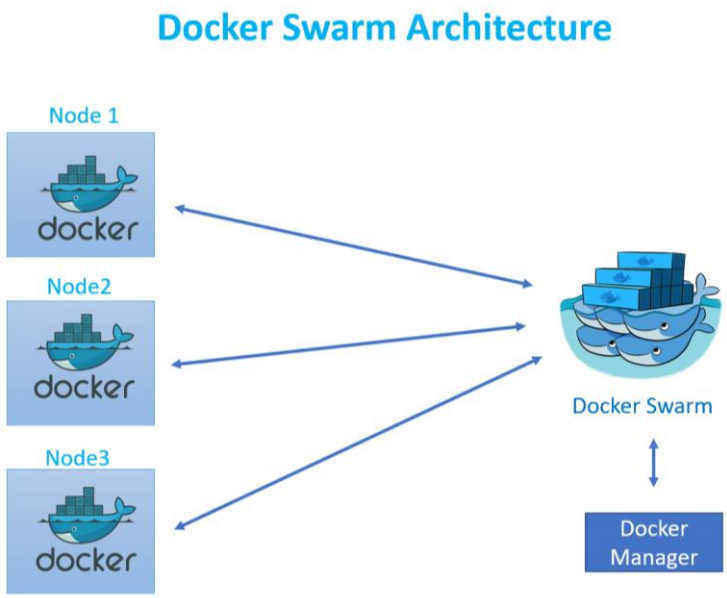
៤.៥. Docker Swarm

Docker Swarm គឺជាមុខងារ Container Orchestration របស់ Docker ដែលអនុញ្ញាតឱ្យគ្រប់គ្រង និងដាក់ឱ្យដំណើរការ Containers នៅលើម៉ាស៊ីនមេច្រើន ដែលត្រូវបានតភ្ជាប់ជាក្រុមហៅថា Swarm Cluster។ បច្ចេកវិទ្យានេះជួយសម្រួលដល់ការគ្រប់គ្រង Services ការចែកចាយ Workloads និងការរក្សាចំនួន Containers ឱ្យស្របតាមការកំណត់។

Docker Swarm មានរចនាសម្ព័ន្ធសំខាន់ពីរប្រភេទ គឺ Manager Nodes និង Worker Nodes។ Manager Nodes ទទួលខុសត្រូវក្នុងការគ្រប់គ្រងស្ថានភាព Cluster និងបែងចែកការងារ ខណៈ Worker Nodes ទទួលភារកិច្ចដំណើរការ Containers តាមការកំណត់របស់ Manager។

ក្នុងគម្រោងនេះ Docker Swarm ត្រូវបានប្រើប្រាស់សម្រាប់បង្កើត Cluster នៅលើ AWS EC2 ដោយមាន Manager Node និង Worker Nodes។ កម្មវិធីត្រូវបានដាក់ឱ្យដំណើរការតាមរយៈ Docker Stack ដែលអនុញ្ញាតឱ្យគ្រប់គ្រង Services, Replicas, Rolling Update និងការចែកចាយសេវាកម្មក្នុង Cluster ។
សមាសភាគសំខាន់ៗរួមមាន៖

- Manager Node៖ ជាម៉ាស៊ីនមេគ្រប់គ្រងស្ថានភាព Cluster និងកំណត់ទីតាំងដំណើរការរបស់ Services។
- Worker Node៖ ជាម៉ាស៊ីនមេដែលទទួល និងដំណើរការ Tasks ឬ Containers តាមការបែងចែកពី Manager។
- Service៖ ជាការកំណត់អំពីកម្មវិធី ឬ Workload ដែលត្រូវដំណើរការក្នុង Cluster។
- Replica៖ ជាចំនួន Instances នៃ Service ដែលត្រូវដំណើរការតាមការកំណត់។
- Stack៖ ជាក្រុមនៃ Services ដែលត្រូវបានកំណត់ និងដាក់ឱ្យដំណើរការរួមគ្នាតាម Stack Configuration។



រូបភាពទី៤.៥៖ Logo Docker Swarm

៤.៦. Amazon Web Services EC2 (AWS EC2)

Amazon Elastic Compute Cloud (EC2) គឺជាសេវាកម្មមួយរបស់ Amazon Web Services (AWS) ដែលផ្តល់ម៉ាស៊ីនមេនិម្មិត (Virtual Servers) សម្រាប់ដំណើរការកម្មវិធី និងសេវាកម្មផ្សេងៗនៅលើ Cloud។ សេវាកម្មនេះអនុញ្ញាតឱ្យអ្នកប្រើប្រាស់ជ្រើសរើសប្រភេទម៉ាស៊ីនមេ ទំហំធនធាន និងការកំណត់បណ្តាញឱ្យសមស្របតាមតម្រូវការរបស់គម្រោង។

ការប្រើប្រាស់ AWS EC2 ជួយកាត់បន្ថយតម្រូវការក្នុងការរៀបចំ និងថែទាំម៉ាស៊ីនមេផ្ទាល់ ដោយអ្នកអភិវឌ្ឍអាចបង្កើត និងគ្រប់គ្រងធនធានតាមរយៈសេវាកម្ម Cloud។ លើសពីនេះ AWS ផ្តល់មុខងារគ្រប់គ្រងបណ្តាញ និងសុវត្ថិភាព ដែលអាចកំណត់ការចូលប្រើម៉ាស៊ីនមេបានតាមតម្រូវការ។

ក្នុងគម្រោងនេះ AWS EC2 ត្រូវបានប្រើប្រាស់ជាហេដ្ឋារចនាសម្ព័ន្ធសម្រាប់បង្កើត Docker Swarm Cluster និងដាក់ឱ្យដំណើរការកម្មវិធី។ Terraform ត្រូវបានប្រើប្រាស់សម្រាប់បង្កើត EC2 Instances ព្រមទាំងកំណត់ Networking, Security Groups, Key Pairs និង Tags ដើម្បីឱ្យការគ្រប់គ្រងធនធានមានភាពងាយស្រួល និងមានរបៀបរៀបរយ។

សមាសភាគសំខាន់ៗរួមមាន៖

- EC2 Instance៖ ជាម៉ាស៊ីនមេនិម្មិតដែលដំណើរការនៅក្នុង AWS Cloud។
- Security Group៖ ជាប្រព័ន្ធច្បាប់ Firewall សម្រាប់គ្រប់គ្រងចរាចរណ៍បណ្តាញចូល និងចេញពីធនធាន។
- Key Pair៖ ជាសោសុវត្ថិភាពដែលប្រើសម្រាប់ផ្ទៀងផ្ទាត់អត្តសញ្ញាណនៅពេលភ្ជាប់ទៅ EC2 តាមរយៈ SSH។
- Tags៖ ជាស្លាកសម្គាល់ធនធាន ដូចជា Name, Role និង Environment ដើម្បីងាយស្រួលក្នុងការគ្រប់គ្រង។



រូបភាពទី៤.៦៖ Logo Amazon EC2

៤.៧. Security និង Secret Management

សុវត្ថិភាពប្រព័ន្ធ គឺជាកត្តាសំខាន់ក្នុងការរៀបចំហេដ្ឋារចនាសម្ព័ន្ធ Cloud និងការអនុវត្ត CI/CD Pipeline។ ក្នុងដំណើរការអភិវឌ្ឍ និងដាក់ឱ្យដំណើរការកម្មវិធី មានព័ត៌មានសម្ងាត់ជាច្រើនដែលត្រូវការការការពារ ដូចជា AWS Access Keys, SSH Private Keys, Docker Registry Credentials និង Telegram Bot Token។

ប្រសិនបើព័ត៌មានសម្ងាត់ទាំងនេះត្រូវបានរក្សាទុកដោយផ្ទាល់នៅក្នុង Source Code ឬ GitHub Repository នោះអាចបង្កឱ្យមានហានិភ័យនៃការលេចធ្លាយព័ត៌មាន និងការចូលប្រើប្រព័ន្ធដោយគ្មានការអនុញ្ញាត។ ដូច្នេះ ការគ្រប់គ្រង Credentials និង Secrets ឱ្យបានត្រឹមត្រូវ គឺជាតម្រូវការចាំបាច់សម្រាប់រក្សាសុវត្ថិភាពនៃគម្រោង។

ក្នុងគម្រោងនេះ Jenkins Credentials ត្រូវបានកំណត់ជាមធ្យោបាយសម្រាប់រក្សាទុក និងប្រើប្រាស់ព័ត៌មានសម្ងាត់ដែលត្រូវការក្នុង CI/CD Pipeline។ ចំណែក Security Groups នៅលើ AWS ត្រូវបានកំណត់ដើម្បីអនុញ្ញាតឱ្យមានតែចរាចរណ៍បណ្តាញ និង Ports ដែលចាំបាច់សម្រាប់ដំណើរការប្រព័ន្ធប៉ុណ្ណោះ។ វិធីសាស្ត្រនេះជួយកាត់បន្ថយហានិភ័យសុវត្ថិភាព និងគាំទ្រដល់ការគ្រប់គ្រងហេដ្ឋារចនាសម្ព័ន្ធ Cloud ប្រកបដោយភាពជឿជាក់។

៤.៨. ការជូនដំណឹងតាម Telegram

Telegram គឺជាកម្មវិធីទំនាក់ទំនងដែលអាចប្រើប្រាស់សម្រាប់ផ្ញើ និងទទួលសារដោយស្វ័យប្រវត្តិតាមរយៈ Telegram Bot។ ក្នុងការអនុវត្ត DevOps ការភ្ជាប់ Telegram Bot ជាមួយ Jenkins អាចជួយឱ្យក្រុមអភិវឌ្ឍទទួលបានព័ត៌មានអំពីស្ថានភាព CI/CD Pipeline បានយ៉ាងឆាប់រហ័ស ដោយមិនចាំបាច់ចូលទៅពិនិត្យ Jenkins ជាប្រចាំ។

ក្នុងគម្រោងនេះ Telegram Bot ត្រូវបានប្រើប្រាស់សម្រាប់ផ្ញើសារជូនដំណឹងទៅកាន់ Telegram Group នៅពេលដំណើរការ Build, Deploy ឬ Rollback បានបញ្ចប់។ សារជូនដំណឹងអាចបង្ហាញព័ត៌មានសំខាន់ៗដូចជា ឈ្មោះគម្រោង លេខ Build កំណែកម្មវិធី សកម្មភាពដែលបានអនុវត្ត និងស្ថានភាពជោគជ័យ ឬបរាជ័យរបស់ Pipeline។

ដើម្បីរក្សាសុវត្ថិភាព Telegram Bot Token ត្រូវរក្សាទុកនៅក្នុង Jenkins Credentials ជំនួសឱ្យការបញ្ចូលដោយផ្ទាល់ទៅក្នុង Jenkinsfile ឬ GitHub Repository។ លើសពីនេះ ត្រូវកំណត់ Chat ID ឱ្យបានត្រឹមត្រូវ ដើម្បីធានាថាសារជូនដំណឹងត្រូវបានផ្ញើទៅកាន់អ្នកប្រើប្រាស់ ឬក្រុមដែលបានកំណត់។

ការជូនដំណឹងតាម Telegram ជួយបង្កើនភាពងាយស្រួលក្នុងការតាមដានលទ្ធផលនៃការអនុវត្ត CI/CD Pipeline និងធ្វើឱ្យសមាជិកក្រុមអាចដឹងពីស្ថានភាពប្រព័ន្ធបានទាន់ពេលវេលា។ ទោះជាយ៉ាងណា ការជូនដំណឹងនេះមិនអាចជំនួសការត្រួតពិនិត្យស្ថានភាព Services, Replicas និងការផ្ទៀងផ្ទាត់ការដំណើរការរបស់កម្មវិធីបន្ទាប់ពី Deploy ឬ Rollback បានទាំងស្រុងនោះទេ។

៥. របៀបកំណត់រចនាសម្ព័ន្ធ និងអនុវត្តគម្រោងតាមដំណាក់កាល

ការអនុវត្តគម្រោង Cloud Infrastructure & DevOps Automation ត្រូវបានរៀបចំជាដំណាក់កាល ដើម្បីធានាថាការអភិវឌ្ឍ ការកំណត់រចនាសម្ព័ន្ធ និងការដាក់ឱ្យដំណើរការកម្មវិធីនៅលើ Cloud មានលំដាប់លំដោយ និងងាយស្រួលក្នុងការគ្រប់គ្រង។ ដំណើរការនេះចាប់ផ្តើមពីការរៀបចំប្រភពកូដ និងការវេចខ្ចប់កម្មវិធីដោយ Docker បន្តទៅការបង្កើតហេដ្ឋារចនាសម្ព័ន្ធ AWS ដោយប្រើ Terraform ការរៀបចំ Docker Swarm Cluster និងការកំណត់ Nginx រហូតដល់ការបង្កើត CI/CD Pipeline ដោយប្រើ Jenkins និងការជូនដំណឹងតាម Telegram។

ដំណាក់កាលនីមួយៗត្រូវបានពិពណ៌នាអំពីគោលបំណង វិធីសាស្ត្រអនុវត្ត ពាក្យបញ្ជាសំខាន់ៗ និងការផ្ទៀងផ្ទាត់លទ្ធផល។ ទិន្នន័យកំណត់រចនាសម្ព័ន្ធមួយចំនួន ដូចជា Repository URL, AWS Region, AMI, Instance Type, Credentials និង IP Address អាចប្រែប្រួលទៅតាមបរិស្ថាន និងការកំណត់ជាក់ស្តែងរបស់គម្រោង។

ជំហានទី ១៖ រៀបចំ Project និង GitHub Repository

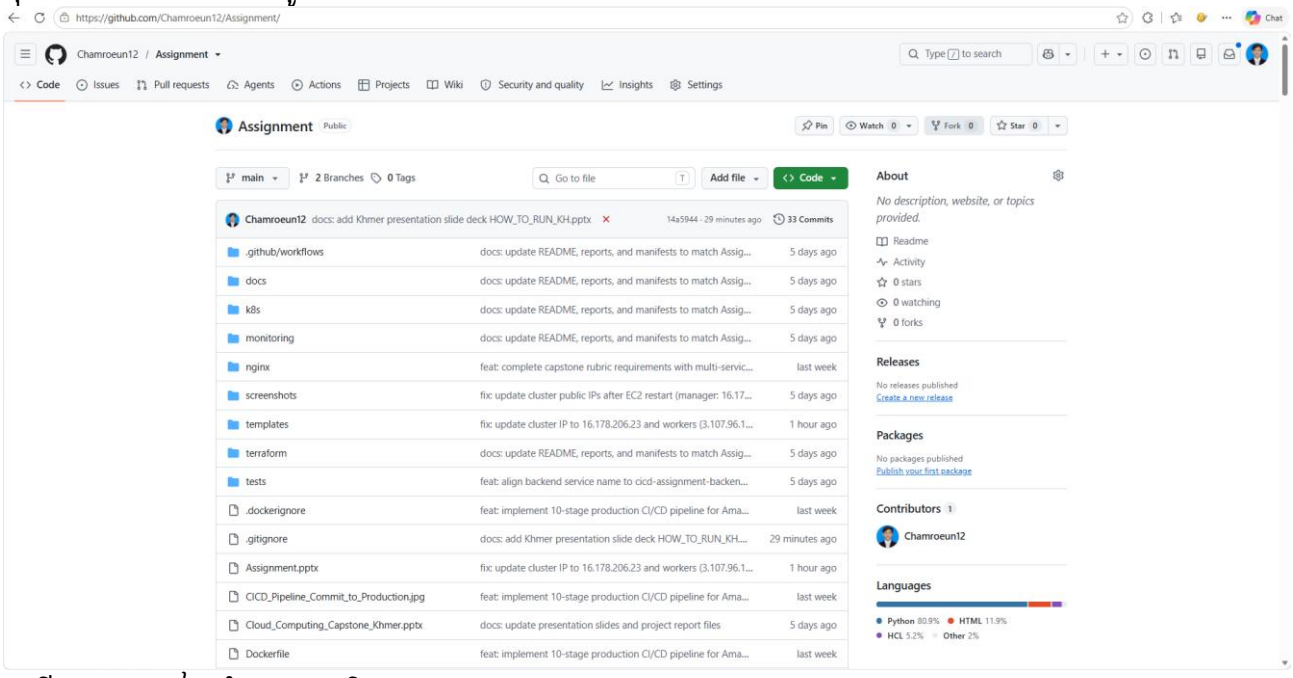
ដំណាក់កាលដំបូងនៃគម្រោង គឺការរៀបចំរចនាសម្ព័ន្ធឯកសារ និងការគ្រប់គ្រងប្រភពកូដ ដើម្បីឱ្យការអភិវឌ្ឍកម្មវិធីមានរបៀបរៀបរយ និងងាយស្រួលក្នុងការតាមដាន។ ក្នុងដំណាក់កាលនេះ Git ត្រូវបានប្រើប្រាស់សម្រាប់គ្រប់គ្រងប្រវត្តិនៃការផ្លាស់ប្តូរកូដ ខណៈ GitHub ត្រូវបានប្រើជាទីតាំងកណ្តាលសម្រាប់រក្សាទុក និងចែករំលែកឯកសាររបស់គម្រោង។

រចនាសម្ព័ន្ធ Project ត្រូវបានបែងចែកជាផ្នែកសំខាន់ៗ ដូចជា Application Source Code, Docker Configuration, Terraform Configuration, Jenkinsfile និង Deployment Files។ ការបែងចែកនេះជួយឱ្យការគ្រប់គ្រង និងការកែប្រែសមាសភាគនីមួយៗមានភាពងាយស្រួល។ ជាមួយគ្នានេះ ឯកសារ `.gitignore` ត្រូវបានកំណត់ ដើម្បីការពារការបញ្ចូលព័ត៌មានសម្ងាត់ ឯកសារ Private Keys, Terraform State និងឯកសារបណ្តោះអាសន្នដែលមិនចាំបាច់ទៅក្នុង Repository។

ពាក្យបញ្ជាសំខាន់ៗសម្រាប់រៀបចំ Git Repository មានដូចខាងក្រោម៖

- git init បង្កើត Git Repository នៅក្នុង Project។
- git add បញ្ចូលឯកសារដែលត្រូវតាមដានទៅក្នុង Staging Area។
- git commit -m Init កត់ត្រាការផ្លាស់ប្តូរដំបូង។
- git branch -M main កំណត់ Branch សំខាន់ឈ្មោះ main។
- git remote add origin https://github.com/Chamroeun12/Assignment.git ភ្ជាប់ Local Repository ទៅ GitHub។
- git push -u origin main បញ្ជូនប្រភពកូដទៅ GitHub Repository។

បន្ទាប់ពីបញ្ជូនប្រភពកូដទៅ GitHub ក្រុមបានរៀបចំការគ្រប់គ្រងកំណែកម្មវិធីតាមរយៈ Git Tags ដូចជា `v1.0.0`, `v1.0.1` និង `v1.1.0`។ ការកំណត់ Tags ជួយសម្គាល់កំណែកម្មវិធីនីមួយៗ និងផ្តល់ភាពងាយស្រួលក្នុងការតាមដានការផ្លាស់ប្តូរ ឬជ្រើសរើសកំណែសម្រាប់ Deployment។



រូបភាពទី៥.១៖ ការរៀបចំ Project និង GitHub Repository

ជំហានទី ២៖ វេចខ្ចប់កម្មវិធីដោយប្រើ Docker (Containerization)

បន្ទាប់ពីរៀបចំ Project និង GitHub Repository រួចរាល់ ជំណាក់កាលបន្ទាប់គឺការវេចខ្ចប់កម្មវិធីទៅជា Docker Image ដើម្បីធ្វើឱ្យកម្មវិធីអាចដំណើរការនៅលើបរិស្ថានផ្សេងៗបានដោយមានភាពស្របគ្នា។

ក្នុងដំណាក់កាលនេះ Dockerfile ត្រូវបានប្រើប្រាស់សម្រាប់កំណត់ការដំឡើង Dependencies ការរៀបចំបរិស្ថានដំណើរការ និងការកំណត់ចាំបាច់ផ្សេងៗរបស់កម្មវិធី។ បន្ទាប់មក Docker Compose ត្រូវបានប្រើដើម្បីបង្កើត Image របស់ Application ដោយផ្អែកលើ Configuration ដែលបានរៀបចំ។

ពាក្យបញ្ជាសម្រាប់បង្កើត Docker Image៖

docker compose build app ក្រោយពេល Build បានជោគជ័យ Docker Image ត្រូវបានបញ្ជូនទៅ Docker Hub ដែលជាទីតាំងរក្សាទុក និងចែកចាយ Images សម្រាប់ប្រើប្រាស់នៅពេល Deployment។

ពាក្យបញ្ជាសម្រាប់បញ្ជូន Image ទៅ Docker Hub៖

- docker push :v1.0.0

តាមលទ្ធផលដែលបានបង្ហាញ Docker Image ត្រូវបាន Build និង Push ទៅកាន់ Docker Hub ដោយមាន Version Tag `v1.0.0`។ ការរក្សាទុក Image នៅក្នុង Container Registry អនុញ្ញាតឱ្យម៉ាស៊ីនមេនៅលើ Cloud អាចទាញយក Image មកប្រើប្រាស់សម្រាប់ដាក់ឱ្យដំណើរការកម្មវិធី។

រូបភាពទី៥.៣៖ លទ្ធផលនៃការបង្កើត Docker Image និងការបញ្ជូនទៅ Docker Hub

ជំហានទី ៣៖ សាកល្បងកម្មវិធីនៅលើ Local ដោយប្រើ Docker Compose

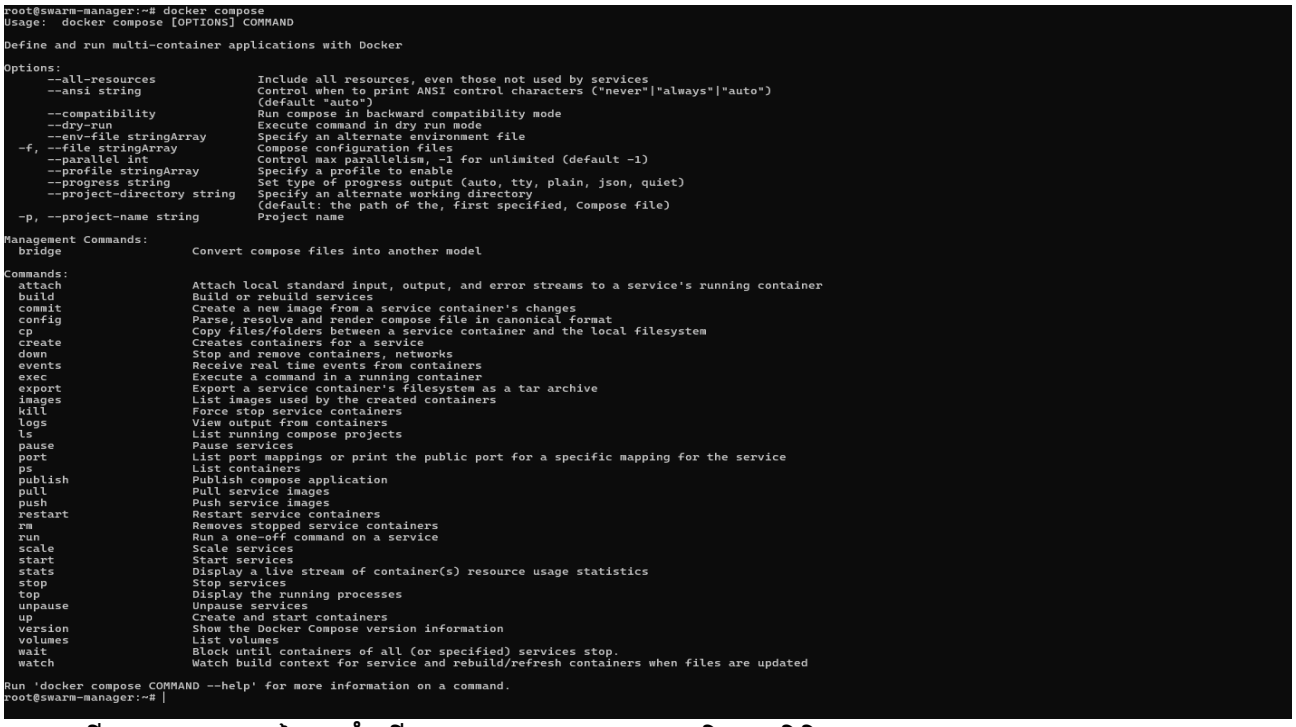
មុនពេលដាក់កម្មវិធីឱ្យដំណើរការនៅលើ Cloud ការសាកល្បងប្រព័ន្ធនៅលើ Local Environment គឺជាដំណាក់កាលសំខាន់ ដើម្បីពិនិត្យមើលការតភ្ជាប់រវាងសេវាកម្ម និងធានាថាការកំណត់រចនាសម្ព័ន្ធមានភាពត្រឹមត្រូវ។

ក្នុងគម្រោងនេះ Docker Compose ត្រូវបានប្រើប្រាស់សម្រាប់គ្រប់គ្រង Multi-container Application ដែលមានសមាសភាគសំខាន់ៗចំនួនបី គឺ Application, Database និង Nginx។ សេវាកម្មទាំងនេះត្រូវបានកំណត់នៅក្នុងឯកសារ docker-compose.yml ដើម្បីអាចដាក់ឱ្យដំណើរការរួមគ្នាបានតាមរយៈពាក្យបញ្ជាតែមួយ។

- ពាក្យបញ្ជាសម្រាប់ដាក់ឱ្យដំណើរការ Containers៖ docker compose up -d
- ពាក្យបញ្ជាសម្រាប់ពិនិត្យស្ថានភាព Containers៖ docker compose ps

លទ្ធផលនៃការសាកល្បងបានបង្ហាញថា Containers សំខាន់ៗរបស់កម្មវិធី រួមមាន `portfolio_app`, `portfolio_db` និង `portfolio_nginx` អាចដំណើរការបាន។ ជាពិសេស Database Container បានបង្ហាញស្ថានភាព Healthy ដែលបញ្ជាក់ថា Health Check របស់សេវាកម្មបានឆ្លងកាត់។

បន្ទាប់មក កម្មវិធីត្រូវបានសាកល្បងតាមរយៈ Web Browser ដោយប្រើអាសយដ្ឋាន `http://localhost:8000`។ លទ្ធផលបង្ហាញថាទំព័រ Portfolio Directory អាចបង្ហាញបាននៅលើ Local Environment។ ការសាកល្បងនេះផ្តល់ភស្តុតាងដំបូងអំពីការដំណើរការរបស់កម្មវិធី មុនពេលបន្តទៅដំណាក់កាល Cloud Deployment។



រូបភាពទី៥.៤៖ លទ្ធផលនៃការដំណើរការ Docker Compose និងការពិនិត្យស្ថានភាព Containers *

រូបភាពទី៥.៥៖ លទ្ធផលសាកល្បងកម្មវិធីនៅលើ Local Environment*

ជំហានទី ៤៖ កំណត់រចនាសម្ព័ន្ធ Terraform សម្រាប់ AWS

ដំណាក់កាលនេះផ្តោតលើការរៀបចំហេដ្ឋារចនាសម្ព័ន្ធ Cloud ដោយប្រើប្រាស់ Terraform តាមគោលការណ៍ Infrastructure as Code (IaC)។ ការប្រើប្រាស់ Terraform អនុញ្ញាតឱ្យក្រុមកំណត់ និងបង្កើតធនធាន AWS តាមរយៈ Configuration Files ជំនួសឱ្យការកំណត់ដោយដៃតាម AWS Management Console។
ដើម្បីធ្វើឱ្យការគ្រប់គ្រងមានរបៀបរៀបរយ ឯកសារ Terraform ត្រូវបានបែងចែកតាមមុខងារផ្សេងៗ ដូចជា៖

- provider.tf៖ កំណត់ Cloud Provider និងការកំណត់ពាក់ព័ន្ធ។
- variables.tf៖ កំណត់ Input Variables សម្រាប់គម្រោង។
- main.tf / network.tf៖ កំណត់ធនធានសំខាន់ៗ និងរចនាសម្ព័ន្ធបណ្តាញ។
- security-group.tf៖ កំណត់ច្បាប់សុវត្ថិភាពបណ្តាញ។
- ec2.tf៖ កំណត់ EC2 Instances សម្រាប់ប្រើប្រាស់ក្នុងគម្រោង។
- outputs.tf៖ បង្ហាញព័ត៌មានធនធានដែលបានបង្កើត ដូចជា Instance IDs និង IP Addresses។

បន្ទាប់ពីរៀបចំ Configuration Files ក្រុមប្រើប្រាស់ពាក្យបញ្ជាសំខាន់ៗរបស់ Terraform ដើម្បីរៀបចំ និងបង្កើតហេដ្ឋារចនាសម្ព័ន្ធ៖

- terraform init
- terraform plan
- terraform apply

ពាក្យបញ្ជា `terraform init` ប្រើសម្រាប់រៀបចំបរិស្ថាន Terraform និង Providers ដែលចាំបាច់។ ចំណែក `terraform plan` ប្រើសម្រាប់ពិនិត្យមើលការផ្លាស់ប្តូរដែលនឹងត្រូវអនុវត្ត ហើយ `terraform apply` ប្រើសម្រាប់បង្កើត ឬកែប្រែធនធានតាម Configuration ដែលបានកំណត់។

តាមលទ្ធផលនៃការអនុវត្ត Terraform បានបង្ហាញព័ត៌មានធនធាន AWS ដែលត្រូវបានបង្កើត រួមមាន VPC, Subnets, Instance IDs និង Public/Private IP Addresses របស់ Worker Nodes។ ការប្រើប្រាស់វិធីសាស្ត្រនេះជួយឱ្យការរៀបចំហេដ្ឋារចនាសម្ព័ន្ធមានភាពស្របគ្នា និងអាចអនុវត្តឡើងវិញបាន។

```
Apply complete! Resources: 3 added, 0 changed, 3 destroyed.

Outputs:

application_health_url = "http://16.178.206.23/health"
application_url = "http://16.178.206.23"
cluster_topology_summary = {
  "architecture" = "1 Manager (Leader) + 2 Workers with Nginx Load Balancer"
  "manager_nodes" = 1
  "region" = "ap-southeast-2"
  "total_nodes" = 3
  "worker_nodes" = 2
}
manager_private_ip = "172.31.34.160"
manager_public_ip = "16.178.206.23"
ssh_connect_manager = "ssh -i <your-private-key.pem> ubuntu@16.178.206.23"
worker_private_ips = [
  "172.31.36.22",
  "172.31.47.183",
]
worker_public_ips = [
  "3.107.96.140",
  "13.211.213.228",
]
```

រូបភាពទី៥.៦៖ លទ្ធផលនៃការប្រើប្រាស់ Terraform ដើម្បីបង្កើតហេដ្ឋារចនាសម្ព័ន្ធ AWS

ជំហានទី ៥៖ ផ្ទៀងផ្ទាត់ AWS EC2 និងរចនាសម្ព័ន្ធបណ្តាញ

បន្ទាប់ពី Terraform បានបង្កើតហេដ្ឋារចនាសម្ព័ន្ធ AWS រួចរាល់ ក្រុមត្រូវធ្វើការត្រួតពិនិត្យធនធានដែលបានបង្កើត ដើម្បីធានាថាចំនួនម៉ាស៊ីនមេ ការកំណត់បណ្តាញ និងស្ថានភាពដំណើរការមានភាពត្រឹមត្រូវតាមតម្រូវការរបស់គម្រោង។

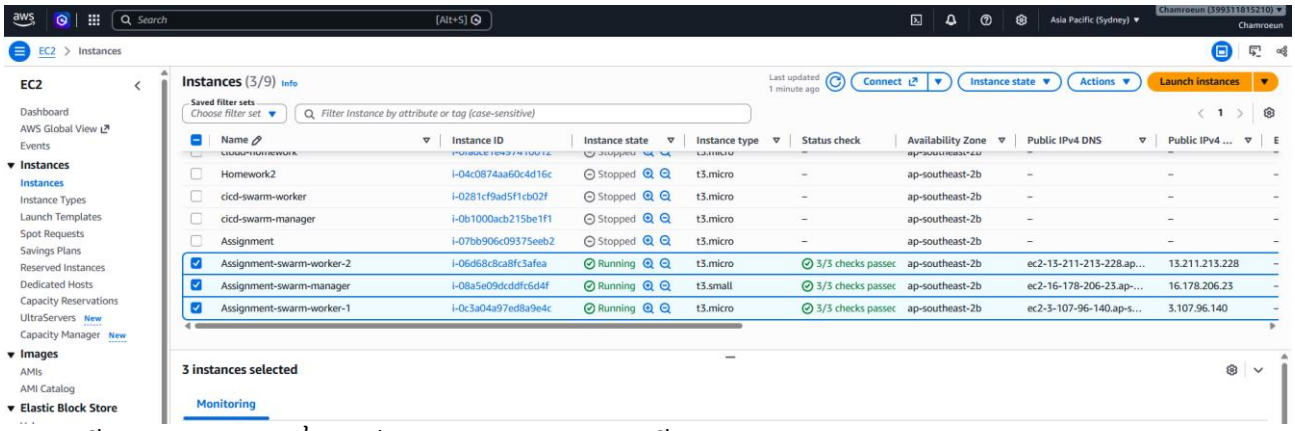
ការផ្ទៀងផ្ទាត់ត្រូវបានធ្វើឡើងតាមរយៈ AWS Management Console ដោយពិនិត្យមើល EC2 Instances, Instance States, Instance Types, Public/Private IP Addresses និង Security Groups។

តាមរូបភាពលទ្ធផល គម្រោងបានរៀបចំ EC2 Instances ចំនួនបី ដែលរួមមាន៖

- swarmmanager៖ ម៉ាស៊ីនមេសម្រាប់គ្រប់គ្រង Docker Swarm Cluster។
- swarmwork1៖ ម៉ាស៊ីនមេ Worker ទីមួយសម្រាប់ដំណើរការ Workloads។
- swarmwork2៖ ម៉ាស៊ីនមេ Worker ទីពីរសម្រាប់ដំណើរការ Workloads។

ម៉ាស៊ីនមេទាំងបីត្រូវបានបង្ហាញក្នុងស្ថានភាព Running និងប្រើប្រាស់ Instance Type `t3.micro`។ លទ្ធផល Status Checks ក៏ត្រូវបានបង្ហាញថាបានឆ្លងកាត់ ដែលជាកស្តុតាងបញ្ជាក់ពីស្ថានភាពដំណើរការរបស់ EC2 Instances នៅពេលត្រួតពិនិត្យ។

លើសពីនេះ ការចូលគ្រប់គ្រងម៉ាស៊ីនមេតាមរយៈ SSH ត្រូវប្រើ Key-based Authentication ជំនួសការរក្សាទុក Password នៅក្នុង Source Code ដើម្បីជួយបង្កើនសុវត្ថិភាពប្រព័ន្ធ។



រូបភាពទី៥.៧៖ លទ្ធផលផ្សៀងផ្ទាត់ EC2 Instances នៅលើ AWS Management Console

ជំហានទី ៦៖ បង្កើត Docker Swarm Cluster

បន្ទាប់ពី EC2 Instances ត្រូវបានបង្កើត និងផ្សៀងផ្ទាត់រួចរាល់ ដំណាក់កាលបន្ទាប់គឺការរៀបចំ Docker Swarm Cluster ដើម្បីអនុញ្ញាតឱ្យកម្មវិធីអាចដំណើរការ និងគ្រប់គ្រងនៅលើម៉ាស៊ីនមេច្រើន។

ក្នុងគម្រោងនេះ Docker Swarm Cluster ត្រូវបានរៀបចំដោយមាន Manager Node ចំនួនមួយ និង Worker Nodes ចំនួនពីរ។ Manager Node មានតួនាទីគ្រប់គ្រងស្ថានភាព Cluster និងបែងចែកការងារទៅកាន់ Nodes ផ្សេងៗ ខណៈ Worker Nodes ទទួលខុសត្រូវក្នុងការដំណើរការ Containers តាមការកំណត់របស់ Manager។

ដំណើរការរៀបចំ Cluster ត្រូវបានគាំទ្រដោយ Terraform និងការអនុវត្ត Remote Commands ដើម្បីកំណត់ Docker Swarm នៅលើ EC2 Instances។ បន្ទាប់ពី Swarm ត្រូវបាន Initialize នៅលើ Manager Node នោះ Worker Nodes ត្រូវបានភ្ជាប់ចូលទៅក្នុង Cluster តាមរយៈ Join Token ដែលបានបង្កើត។

តាមលទ្ធផលដែលបានបង្ហាញ Docker Swarm Cluster មាន Nodes សំខាន់ៗចំនួនបី គឺ `swarmmanager`, `swarmwork1` និង `swarmwork2`។ Nodes ទាំងបីបានបង្ហាញស្ថានភាព Ready និង Active ខណៈ Manager Node ត្រូវបានសម្គាល់ថាមានតួនាទីជា Leader។

លទ្ធផលនេះបញ្ជាក់ថា Docker Swarm Cluster ត្រូវបានរៀបចំ និងអាចស្គាល់ Nodes ដែលបានភ្ជាប់ចូលក្នុងប្រព័ន្ធ។ ទោះជាយ៉ាងណា ព័ត៌មានសម្ងាត់ដូចជា Swarm Join Token មិនត្រូវរក្សាទុកនៅក្នុង Public Repository ឡើយ។

```
ubuntu@swarm-manager:~$ docker node ls
ID                            HOSTNAME        STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
reszp0cx4eshquabpqwary0n2 *   swarm-manager   Ready     Active         Leader           29.9.3
yz5suzv7h7968ql6ftyetjtbe     swarm-worker1   Ready     Active                          29.9.3
nxtmcwyif4sc6g3qd2q81nn2a     swarm-worker2   Ready     Active                          29.9.3
ubuntu@swarm-manager:~$
```

រូបភាពទី៥.៨៖ លទ្ធផលបង្កើត Docker Swarm Cluster ដោយប្រើ Terraform

ជំហានទី ៧៖ រៀបចំ Docker Stack សម្រាប់ Deployment

បន្ទាប់ពី Docker Swarm Cluster បានដំណើរការ ដំណាក់កាលបន្ទាប់គឺការរៀបចំ Docker Stack ដើម្បីដាក់ឱ្យដំណើរការសេវាកម្មរបស់កម្មវិធីនៅលើ Cluster។

ក្នុងដំណាក់កាលនេះ ឯកសារ Docker Compose ដែលប្រើជាមួយ Docker Stack ត្រូវបានរៀបចំឱ្យមានការកំណត់សំខាន់ៗ ដូចជា Services, Images, Networks, Replicas, Restart Policies និង Update Configurations។ ការកំណត់ទាំងនេះមានតួនាទីកំណត់ចំនួន Containers ដែលត្រូវដំណើរការ និងរបៀបគ្រប់គ្រងសេវាកម្មនៅក្នុង Cluster។

តាមលទ្ធផលនៃការអនុវត្ត Docker Stack បានបង្ហាញសេវាកម្មសំខាន់ៗចំនួនបី រួមមាន៖

- portfolio_app៖ សេវាកម្មកម្មវិធីដែលបានកំណត់ Replicas ចំនួន ៣។
- portfolio_db៖ សេវាកម្ម Database ដែលបានកំណត់ Replica ចំនួន ១។
- portfolio_nginx៖ សេវាកម្ម Web Server និង Reverse Proxy ដែលបានកំណត់ Replicas ចំនួន ២។

លទ្ធផលបានបង្ហាញចំនួន Replicas របស់ Services ទាំងបីជា `3/3`, `1/1` និង `2/2` ដែលមានន័យថាចំនួន Tasks ដែលកំពុងដំណើរការស្របតាមចំនួនដែលបានកំណត់នៅពេលពិនិត្យ។

ការដាក់ឱ្យដំណើរការតាម Docker Stack ជួយឱ្យសេវាកម្មរបស់កម្មវិធីអាចគ្រប់គ្រងនៅក្នុង Cluster តែមួយ និងផ្តល់ភាពងាយស្រួលសម្រាប់ការធ្វើបច្ចុប្បន្នភាព និងការត្រួតពិនិត្យស្ថានភាព Services។

```
ubuntu@swarm-manager:~$ docker service ls
ID             NAME                    MODE         REPLICAS   IMAGE                                                                          PORTS
ciqx8p28be3s   cicd-assignment_nginx   replicated   1/1        nginx:alpine                                                                   *:80->80/tcp
slu8xzej31i7   cicd-assignment_web     replicated   2/2        399311815210.dkr.ecr.ap-southeast-2.amazonaws.com/cicd-assignment:latest
ubuntu@swarm-manager:~$
```

រូបភាពទី៥.៩៖ លទ្ធផល Docker Stack Deployment និងស្ថានភាព Services/Replicas

ជំហានទី ៨៖ កំណត់ Nginx ជា Load Balancer និង Reverse Proxy

Nginx ត្រូវបានប្រើប្រាស់ជាសមាសភាគសំខាន់សម្រាប់គ្រប់គ្រងការទទួល និងបញ្ជូនសំណើរបស់អ្នកប្រើប្រាស់ទៅកាន់សេវាកម្មកម្មវិធីនៅខាងក្រោយ។ ក្នុងគម្រោងនេះ Nginx ត្រូវបានកំណត់ឱ្យមានតួនាទីជា Reverse Proxy និង Load Balancer ដើម្បីគាំទ្រការចូលប្រើកម្មវិធីដែលដំណើរការនៅក្នុង Docker Swarm Cluster។

នៅពេលអ្នកប្រើប្រាស់ផ្ញើ HTTP Request ទៅកាន់ Public IP Address របស់ប្រព័ន្ធ Nginx ទទួលសំណើនោះ ហើយបញ្ជូនបន្តទៅកាន់ Backend Service តាម Configuration ដែលបានកំណត់។

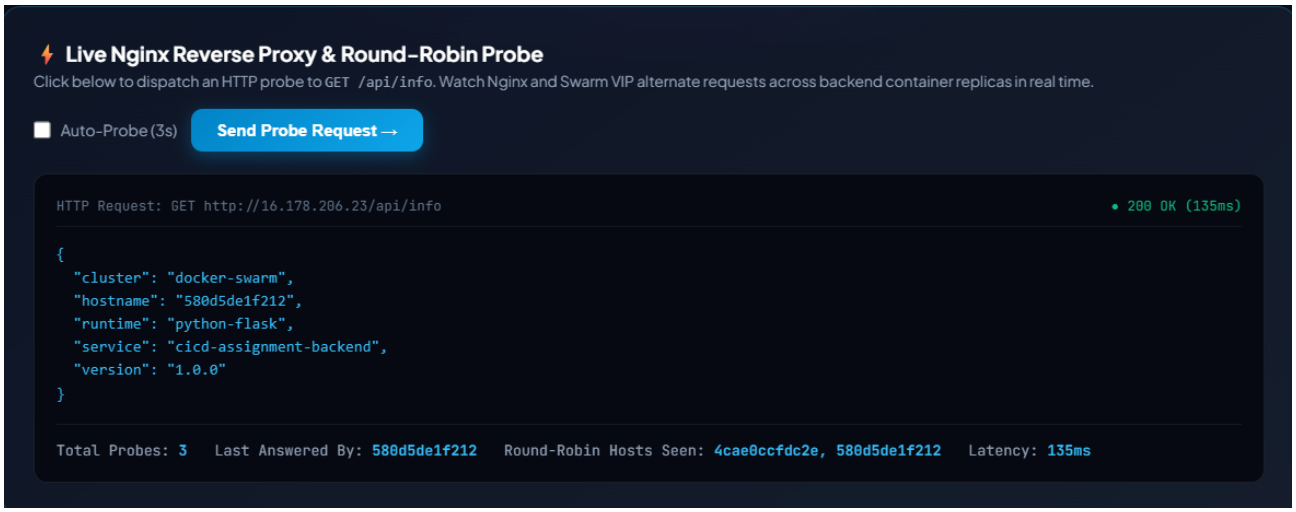
បន្ទាប់ពីរៀបចំ Nginx រួចរាល់ ក្រុមបានប្រើប្រាស់ពាក្យបញ្ជា `curl` ដើម្បីសាកល្បងការឆ្លើយតបរបស់ Web Server និងការចូលប្រើសេវាកម្ម។

ឧទាហរណ៍ពាក្យបញ្ជាសម្រាប់ផ្ទៀងផ្ទាត់ HTTP Response៖

`curl -I http://<EC2_PUBLIC_IP>`

`curl http://<EC2_PUBLIC_IP>/som`

តាមរូបភាពសាកល្បង ប្រព័ន្ធបានផ្តល់ HTTP Response Headers ដែលជាភស្តុតាងបង្ហាញថាមានការឆ្លើយតបពី Web Server។ ការត្រួតពិនិត្យនេះជួយផ្ទៀងផ្ទាត់ការតភ្ជាប់រវាង Client, Nginx និង Backend Services មុនពេលបន្តទៅការរៀបចំ CI/CD Pipeline។



រូបភាពទី៥.១០៖ លទ្ធផលសាកល្បង Nginx Reverse Proxy និង HTTP Response

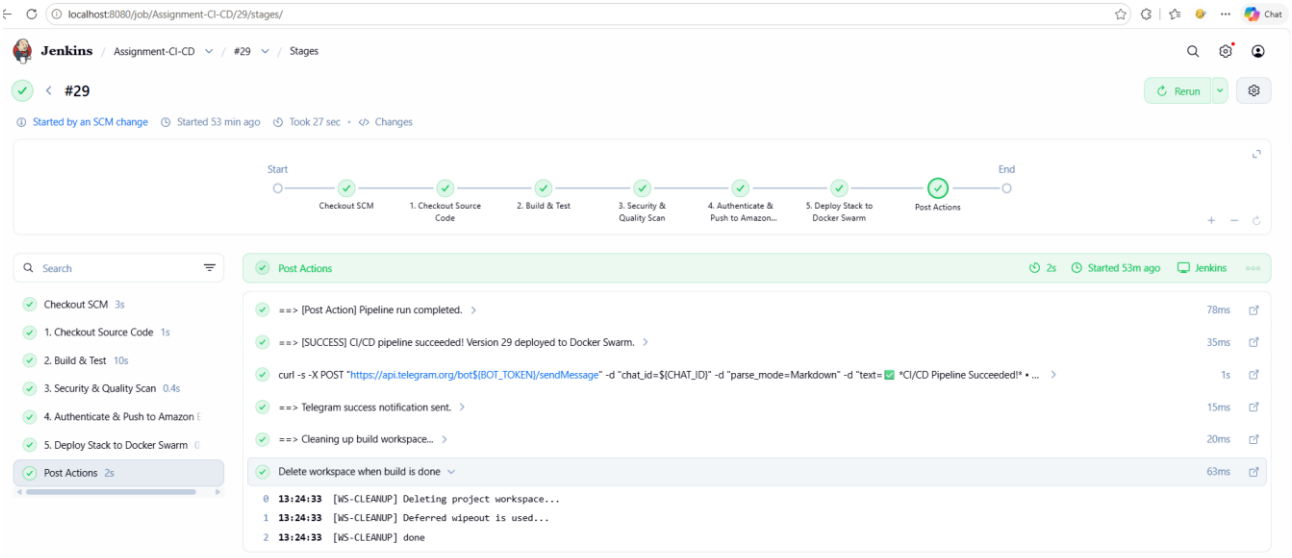
ជំហានទី ៩៖ ភ្ជាប់ GitHub ជាមួយ Jenkins តាមរយៈ Webhook

ដើម្បីធ្វើឱ្យការអភិវឌ្ឍ និងដាក់ឱ្យដំណើរការកម្មវិធីមានលក្ខណៈស្វ័យប្រវត្តិ GitHub Repository ត្រូវបានភ្ជាប់ជាមួយ Jenkins តាមរយៈ GitHub Webhook។ ការភ្ជាប់នេះអនុញ្ញាតឱ្យ GitHub ផ្ញើការជូនដំណឹងទៅ Jenkins នៅពេលមានព្រឹត្តិការណ៍ដែលបានកំណត់ ដូចជាការបញ្ជូនប្រភពកូដថ្មី (Push) ទៅ Repository។

ក្នុងដំណាក់កាលនេះ Webhook ត្រូវបានកំណត់នៅក្នុង GitHub Repository ដោយបញ្ជាក់ URL សម្រាប់ទទួលការជូនដំណឹងរបស់ Jenkins និងជ្រើសរើសប្រភេទ Events ដែលត្រូវប្រើសម្រាប់ចាប់ផ្តើម Pipeline។

នៅពេល GitHub ទទួលបានការផ្លាស់ប្តូរកូដ វាអាចផ្ញើ HTTP POST Request ទៅ Jenkins ដើម្បីជូនដំណឹងអំពីការផ្លាស់ប្តូរនោះ។ ប្រសិនបើ Jenkins Job បានកំណត់ Trigger ត្រឹមត្រូវ Pipeline នឹងអាចចាប់ផ្តើមដំណើរការដោយស្វ័យប្រវត្តិ។

តាមរូបភាព GitHub Webhook បានបង្ហាញស្ថានភាពថា Last Delivery Was Successful ដែលបញ្ជាក់ថាការផ្ញើ Webhook Request បានជោគជ័យ។ លទ្ធផលនេះជាភស្តុតាងមួយសម្រាប់ការតភ្ជាប់ GitHub ជាមួយប្រព័ន្ធទទួល Webhook។



រូបភាពទី៥.១១៖ ការកំណត់ GitHub Webhook សម្រាប់ភ្ជាប់ជាមួយ Jenkins

ជំហានទី ១០៖ រៀបចំ Jenkins CI/CD Pipeline

Jenkins CI/CD Pipeline គឺជាផ្នែកសំខាន់នៃគម្រោងដែលប្រើសម្រាប់គ្រប់គ្រងដំណើរការបង្កើត សាកល្បង និងដាក់ឱ្យដំណើរការកម្មវិធីតាមលំដាប់ដែលបានកំណត់។

ក្នុងដំណាក់កាលនេះ ឯកសារ Jenkinsfile ត្រូវបានរៀបចំនៅក្នុង GitHub Repository ដើម្បីកំណត់ Pipeline Stages និងសកម្មភាពដែលត្រូវអនុវត្ត។ ដំណើរការរបស់ Pipeline ត្រូវបានរៀបចំឱ្យមានការត្រួតពិនិត្យស្ថានភាពក្នុងដំណាក់កាលនីមួយៗ ដោយដំណាក់កាល Deployment ត្រូវអនុវត្តនៅពេលលក្ខខណ្ឌ និងការធ្វើតេស្តចាំបាច់បានជោគជ័យ។

លំហូរការងារ CI/CD Pipeline រួមមាន៖

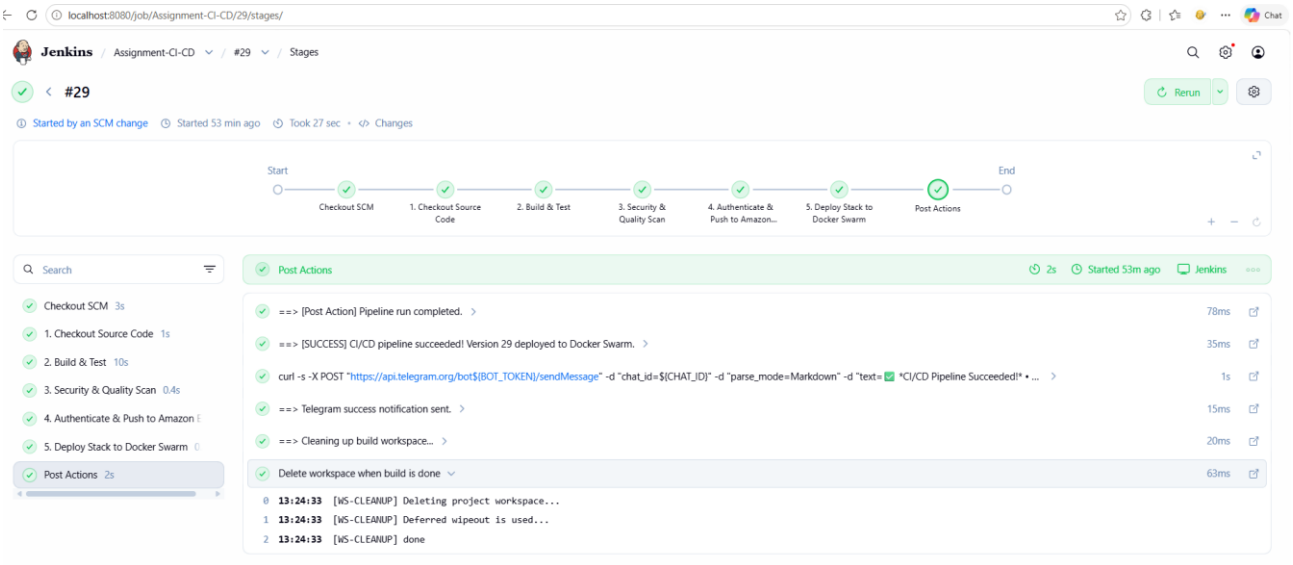
* *Checkout → Build → Test → Push Image → Deploy to Swarm → Verify → Notification * *

ដំណាក់កាល Checkout ទាញយកប្រភពកូដពី GitHub ខណៈ Build ប្រើសម្រាប់បង្កើតកម្មវិធី ឬ Docker Image។ បន្ទាប់មក Test ត្រូវបានប្រើដើម្បីផ្ទៀងផ្ទាត់លទ្ធផលមុនបញ្ជូន Image ទៅ Container Registry។ នៅដំណាក់កាល Deploy កម្មវិធីត្រូវបានដាក់ឱ្យដំណើរការនៅលើ Docker Swarm ហើយ Verify ត្រូវបានប្រើសម្រាប់ត្រួតពិនិត្យស្ថានភាពកម្មវិធីក្រោយ Deployment។

ដើម្បីរក្សាសុវត្ថិភាព ព័ត៌មានសម្ងាត់ដែលត្រូវប្រើក្នុង Pipeline ដូចជា Docker Registry Credentials, SSH Keys និង Telegram Bot Token ត្រូវបានគ្រប់គ្រងតាមរយៈ Jenkins Credentials ជំនួសឱ្យការរក្សាទុកដោយផ្ទាល់ក្នុង Jenkinsfile។

តាមលទ្ធផលដែលបានបង្ហាញ Jenkins Pipeline មានការបែងចែកដំណាក់កាលយ៉ាងច្បាស់លាស់ និងមានមុខងារសម្រាប់គ្រប់គ្រងសកម្មភាព Build, Deploy និង Rollback។ រូបភាព Jenkins Console Output បានបង្ហាញថា Pipeline សម្រាប់សកម្មភាព Deploy កំណែ `v1.0.0` បានបញ្ចប់ដោយស្ថានភាព `SUCCESS`។

បន្ទាប់ពីដាក់ឱ្យដំណើរការ កម្មវិធីក៏ត្រូវបានបង្ហាញតាមរយៈ Web Browser នៅលើ AWS EC2 ដែលជាកស្តតាងបន្ថែមនៃការចូលប្រើកម្មវិធីក្រោយ Deployment។



រូបភាពទី៥.១២៖ លទ្ធផលដំណើរការ Jenkins CI/CD Pipeline

ជំហានទី ១១៖ ការធ្វើ Rolling Update និង Rollback

ការគ្រប់គ្រងកំណែកម្មវិធីជាផ្នែកសំខាន់មួយក្នុងដំណើរការ Deployment ព្រោះកម្មវិធីអាចត្រូវការការធ្វើបច្ចុប្បន្នភាព ដើម្បីបន្ថែមមុខងារថ្មី ឬកែប្រែកំហុស។ ក្នុងគម្រោងនេះ Docker Swarm និង Jenkins ត្រូវបានប្រើសម្រាប់គាំទ្រការផ្លាស់ប្តូរកំណែកម្មវិធី និងការគ្រប់គ្រងដំណើរការ Deployment។

Rolling Update គឺជាវិធីសាស្ត្រធ្វើបច្ចុប្បន្នភាពកម្មវិធីជាបន្តបន្ទាប់ ដោយផ្លាស់ប្តូរ Service Tasks ទៅកំណែថ្មីតាមការកំណត់។ វិធីសាស្ត្រនេះមានគោលបំណងកាត់បន្ថយការរំខានដល់សេវាកម្មក្នុងអំឡុងពេលធ្វើបច្ចុប្បន្នភាព។

ចំណែក Rollback គឺជាដំណើរការត្រឡប់កម្មវិធីទៅកំណែមុន ក្នុងករណីដែលកំណែថ្មីមានបញ្ហា ឬមិនអាចដំណើរការតាមការរំពឹងទុក។ ការអនុវត្ត Rollback ជួយឱ្យក្រុមអាចគ្រប់គ្រងការផ្លាស់ប្តូរកំណែកម្មវិធីបានកាន់តែងាយស្រួល។

ក្នុងដំណាក់កាលនេះ ការផ្លាស់ប្តូរកំណែកម្មវិធីអាចត្រូវបានអនុវត្តតាមរយៈការកំណត់ Docker Image Tag និងការដាក់ឱ្យដំណើរការ Stack ឡើងវិញ។ បន្ទាប់ពីការផ្លាស់ប្តូរ ត្រូវពិនិត្យស្ថានភាព Services និង Replicas ដើម្បីធ្វើឱ្យផ្ទៀងផ្ទាត់ថាប្រព័ន្ធកំពុងដំណើរការតាមការកំណត់។

ជំហានទី ១២៖ កំណត់ការជូនដំណឹងតាម Telegram

ដំណាក់កាលចុងក្រោយនៃគម្រោង គឺការរៀបចំប្រព័ន្ធជូនដំណឹងតាម Telegram ដើម្បីឱ្យសមាជិកក្រុមអាចទទួលបានព័ត៌មានអំពីស្ថានភាពនៃ Jenkins CI/CD Pipeline បានឆាប់រហ័ស។

ក្នុងដំណាក់កាលនេះ Telegram Bot ត្រូវបានបង្កើតតាមរយៈ BotFather ហើយ Chat ID ត្រូវបានកំណត់សម្រាប់អ្នកប្រើប្រាស់ ឬ Telegram Group ដែលត្រូវទទួលសារជូនដំណឹង។ បន្ទាប់មក Telegram Bot ត្រូវបានភ្ជាប់ជាមួយ Jenkins ដើម្បីអនុញ្ញាតឱ្យ Pipeline ផ្ញើសារជូនដំណឹងដោយស្វ័យប្រវត្តិនៅពេលសកម្មភាព Build, Deploy ឬ Rollback បានបញ្ចប់។

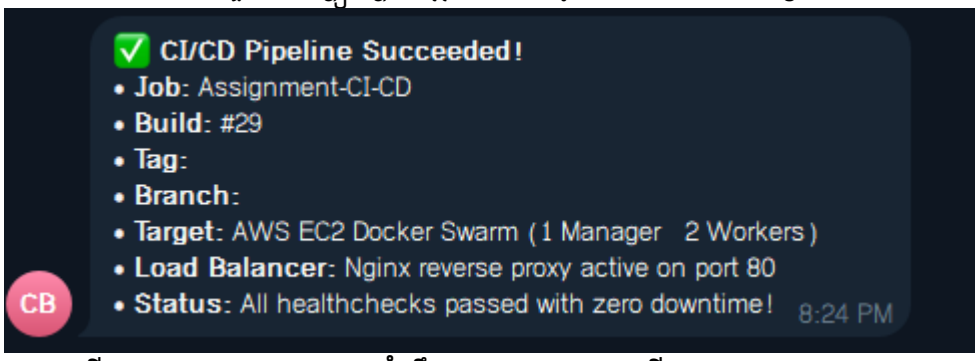
ព័ត៌មានដែលអាចបង្ហាញក្នុងសារជូនដំណឹងរួមមាន៖

- ឈ្មោះ Jenkins Job។
- លេខ Build របស់ Pipeline។
- ប្រភេទសកម្មភាព ដូចជា Build, Deploy ឬ Rollback។
- កំណែកម្មវិធីដែលបានប្រើប្រាស់។
- ស្ថានភាពជោគជ័យ ឬបរាជ័យនៃ Pipeline។

ដើម្បីការពារព័ត៌មានសម្ងាត់ Telegram Bot Token ត្រូវរក្សាទុកនៅក្នុង Jenkins Credentials និងមិនត្រូវបញ្ចូលដោយផ្ទាល់ទៅក្នុង GitHub Repository ឬ Jenkinsfile ឡើយ។

តាមលទ្ធផល Jenkins Console Output បានបង្ហាញសារ `Telegram notification sent successfully.` ដែលបញ្ជាក់ថាដំណើរការផ្ញើសារជូនដំណឹងត្រូវបានរាយការណ៍ថាជោគជ័យ។ លើសពីនេះ Pipeline សម្រាប់សកម្មភាព Deploy កំណែ `v1.1.0` ក៏បានបង្ហាញស្ថានភាព `SUCCESS` ផងដែរ។

ការជូនដំណឹងតាម Telegram ជួយឱ្យក្រុមអភិវឌ្ឍអាចតាមដានលទ្ធផលការងារបានទាន់ពេលវេលា និងកាត់បន្ថយតម្រូវការចូលទៅពិនិត្យ Jenkins ដោយផ្ទាល់ជាប្រចាំ។ ទោះជាយ៉ាងណា ការទទួលបានសារជូនដំណឹងជោគជ័យមិនអាចជំនួសការផ្ទៀងផ្ទាត់ស្ថានភាពកម្មវិធី និង Services ក្រោយ Deployment បានទាំងស្រុងនោះទេ។



រូបភាពទី៥.១៦៖ លទ្ធផលការជូនដំណឹងតាម Telegram ពី Jenkins CI/CD Pipeline

សេចក្តីសង្ខេបនៃដំណើរការអនុវត្ត

តាមរយៈដំណាក់កាលទាំង ១២ ខាងលើ គម្រោងបានបង្ហាញពីដំណើរការរួមបញ្ចូលបច្ចេកវិទ្យា Cloud Computing និង DevOps ចាប់ពីការគ្រប់គ្រងប្រភពកូដនៅលើ GitHub ការវេចខ្ចប់កម្មវិធីដោយ Docker ការរៀបចំ

ហេដ្ឋារចនាសម្ព័ន្ធ AWS ដោយ Terraform និងការគ្រប់គ្រង Services នៅលើ Docker Swarm រហូតដល់ការអនុវត្ត CI/CD Pipeline តាមរយៈ Jenkins។

ការកំណត់ Nginx ការគ្រប់គ្រងកំណែកម្មវិធី និងការជូនដំណឹងតាម Telegram ក៏បានចូលរួមគាំទ្រដល់ដំណើរការដាក់ឱ្យដំណើរការ និងតាមដានលទ្ធផលរបស់ប្រព័ន្ធផងដែរ។ កស្តុតាងដែលបានកត់ត្រាក្នុងដំណាក់កាលនីមួយៗ អាចប្រើប្រាស់សម្រាប់ពិនិត្យ និងវាយតម្លៃលទ្ធផលនៃការអនុវត្តគម្រោង ព្រមទាំងកំណត់ចំណុចដែលត្រូវការកែលម្អបន្ថែមនាពេលអនាគត។

៦. លទ្ធផល

បន្ទាប់ពីបានអនុវត្តគម្រោង Cloud Infrastructure & DevOps Automation តាមដំណាក់កាលដែលបានរៀបរាប់ក្នុងផ្នែកទី៥ ក្រុមបានទទួលលទ្ធផលជាក់ស្តែងពីការរួមបញ្ចូលបច្ចេកវិទ្យា Cloud Computing និង DevOps ដើម្បីរៀបចំហេដ្ឋារចនាសម្ព័ន្ធ និងដាក់ឱ្យដំណើរការកម្មវិធីនៅលើ Cloud។ លទ្ធផលនៃការអនុវត្តត្រូវបានកត់ត្រាតាមរយៈរូបភាព ពាក្យបញ្ជា និងស្ថានភាពប្រព័ន្ធ ដែលបង្ហាញពីដំណើរការចាប់ពីការគ្រប់គ្រងប្រភពកូដ រហូតដល់ការដាក់ឱ្យដំណើរការ ការគ្រប់គ្រងកំណែកម្មវិធី និងការជូនដំណឹងអំពីលទ្ធផល CI/CD Pipeline។

៦.១. លទ្ធផលនៃការគ្រប់គ្រងប្រភពកូដ និង Docker

ការរៀបចំប្រភពកូដរបស់គម្រោងត្រូវបានអនុវត្តដោយប្រើ Git និង GitHub ដែលជួយឱ្យការរក្សាទុកឯកសារ និងការតាមដានការផ្លាស់ប្តូរកម្មវិធីមានរបៀបរៀបរយ។ GitHub Repository ត្រូវបានរៀបចំដោយមានការបែងចែកឯកសារតាមមុខងារផ្សេងៗ ហើយការគ្រប់គ្រងកំណែកម្មវិធីត្រូវបានអនុវត្តតាមរយៈ Git Tags ដូចជា `v1.0.0`, `v1.0.1` និង `v1.1.0`។ លទ្ធផលនេះត្រូវបានបង្ហាញនៅក្នុងរូបភាពទី៥.១ និងទី៥.២។

ចំពោះការវេចខ្ចប់កម្មវិធី Docker ត្រូវបានប្រើប្រាស់សម្រាប់បង្កើត Docker Image និងបញ្ជូនទៅកាន់ Docker Hub ដើម្បីឱ្យអាចទាញយកទៅប្រើប្រាស់នៅលើម៉ាស៊ីនមេ Cloud។ លទ្ធផលនៃការប្រើប្រាស់ពាក្យបញ្ជា `docker compose build` និង `docker push` បានបង្ហាញថា Docker Image កំណែ `v1.0.0` ត្រូវបានបង្កើត និងបញ្ជូនទៅកាន់ Docker Hub ដោយជោគជ័យ។

បន្ថែមពីនេះ ការសាកល្បងកម្មវិធីនៅលើ Local Environment ដោយប្រើ Docker Compose បានបង្ហាញថា Services សំខាន់ៗ រួមមាន Application, Database និង Nginx អាចដំណើរការរួមគ្នាបាន។ លទ្ធផលពិនិត្យ Containers បានបង្ហាញស្ថានភាពដំណើរការរបស់សេវាកម្ម ហើយកម្មវិធីក៏អាចចូលប្រើបានតាមរយៈ Web Browser នៅអាសយដ្ឋាន `http://localhost:8000`។ កស្តុតាងទាំងនេះត្រូវបានបង្ហាញនៅក្នុងរូបភាពទី៥.៣ ដល់ទី៥.៥។

៦.២. លទ្ធផលនៃការបង្កើតហេដ្ឋារចនាសម្ព័ន្ធ AWS និង Docker Swarm

ការរៀបចំហេដ្ឋារចនាសម្ព័ន្ធ Cloud ត្រូវបានអនុវត្តដោយប្រើ Terraform ដើម្បីបង្កើតធនធាននៅលើ Amazon Web Services (AWS) តាមគោលការណ៍ Infrastructure as Code (IaC)។ លទ្ធផលនៃការអនុវត្ត Terraform បានបង្ហាញព័ត៌មានធនធានដែលបានបង្កើត រួមមាន VPC, Subnets, EC2 Instances និង Public/Private IP Addresses។

តាមការផ្ទៀងផ្ទាត់នៅលើ AWS Management Console គម្រោងបានបង្កើត EC2 Instances ចំនួនបី ដែលរួមមាន Manager Node ចំនួនមួយ និង Worker Nodes ចំនួនពីរ។ ម៉ាស៊ីនមេទាំងបីបានបង្ហាញស្ថានភាព Running ហើយការត្រួតពិនិត្យ Status Checks ក៏បានបង្ហាញលទ្ធផលជោគជ័យផងដែរ។

បន្ទាប់ពីរៀបចំ EC2 Instances រួចរាល់ Docker Swarm Cluster ត្រូវបានបង្កើតឡើង ដើម្បីគ្រប់គ្រងការដំណើរការ Containers នៅលើម៉ាស៊ីនមេច្រើន។ លទ្ធផលនៃការត្រួតពិនិត្យ Cluster បានបង្ហាញថា Nodes ទាំងបីគឺ `swarmmanager`, `swarmwork1` និង `swarmwork2` មានស្ថានភាព Ready និង Active ខណៈ Manager Node មានតួនាទីជា Leader។

សម្រាប់ការដាក់ឱ្យដំណើរការកម្មវិធីតាមរយៈ Docker Stack លទ្ធផលបានបង្ហាញថា Services សំខាន់ៗអាចដំណើរការតាមចំនួន Replicas ដែលបានកំណត់។ នៅក្នុងលទ្ធផលផ្ទៀងផ្ទាត់ចុងក្រោយ Services រួមមាន `portfolio_app` ចំនួន `3/3`, `portfolio_db` ចំនួន `1/1` និង `portfolio_nginx` ចំនួន `3/3`។ លទ្ធផលនេះបង្ហាញថាចំនួន Service Tasks ដែលកំពុងដំណើរការស្របតាមចំនួនដែលបានកំណត់នៅពេលត្រួតពិនិត្យ។

ភស្តុតាងនៃការបង្កើតហេដ្ឋារចនាសម្ព័ន្ធ និងការរៀបចំ Docker Swarm ត្រូវបានបង្ហាញនៅក្នុងរូបភាពទី៥.៦ ដល់ទី៥.៩ ព្រមទាំងរូបភាពទី៦.១។

៦.៣. លទ្ធផលនៃការកំណត់ Nginx និងការចូលប្រើកម្មវិធី

បន្ទាប់ពីដាក់ឱ្យដំណើរការ Services នៅលើ Docker Swarm Cluster ក្រុមបានរៀបចំ Nginx ដើម្បីបំពេញតួនាទីជា Reverse Proxy និង Load Balancer សម្រាប់ទទួល និងបញ្ជូន HTTP Requests ទៅកាន់ Backend Services។

ការសាកល្បងត្រូវបានអនុវត្តដោយប្រើពាក្យបញ្ជា `curl` ដើម្បីពិនិត្យ HTTP Response ពី Web Server។ លទ្ធផលដែលបានកត់ត្រាបង្ហាញថា Server អាចឆ្លើយតបទៅនឹងសំណើ HTTP ហើយកម្មវិធីក៏អាចចូលប្រើតាមរយៈ Public IP Address របស់ AWS EC2 បានផងដែរ។

បន្ថែមពីនេះ ការសាកល្បងតាមរយៈ Web Browser បានបង្ហាញទំព័ររបស់កម្មវិធី Portfolio ដែលជាភស្តុតាងបញ្ជាក់ថា Web Application អាចបង្ហាញដល់អ្នកប្រើប្រាស់តាមបណ្តាញបាន។ លទ្ធផលនេះបង្ហាញពីការតភ្ជាប់រវាងហេដ្ឋារចនាសម្ព័ន្ធ Cloud, Docker Services និង Web Server ក្នុងដំណើរការ Deployment។

ភស្តុតាងនៃការសាកល្បង HTTP Response និងការចូលប្រើកម្មវិធីត្រូវបានបង្ហាញនៅក្នុងរូបភាពទី៥.១០ ទី ៥.១៤ និងទី៦.១។

៦.៤. លទ្ធផលនៃការអនុវត្ត Jenkins CI/CD Pipeline

ការអនុវត្ត Jenkins CI/CD Pipeline គឺជាលទ្ធផលសំខាន់មួយនៃគម្រោង ដោយបានបង្ហាញពីការភ្ជាប់ដំណើរការគ្រប់គ្រងប្រភពកូដ ការបង្កើតកម្មវិធី និងការដាក់ឱ្យដំណើរការកម្មវិធីនៅលើ Cloud។

GitHub ត្រូវបានភ្ជាប់ជាមួយ Jenkins តាមរយៈ Webhook ដើម្បីគាំទ្រការជូនដំណឹងអំពីការផ្លាស់ប្តូរប្រភពកូដ។ លទ្ធផលនៅក្នុង GitHub បានបង្ហាញថាការផ្ញើ Webhook Request ទៅកាន់ Endpoint ដែលបានកំណត់មានស្ថានភាពជោគជ័យ។

ក្នុង Jenkins ក្រុមបានរៀបចំ CI/CD Pipeline ដែលមានដំណាក់កាលសំខាន់ៗ ដូចជា Checkout, Build, Test, Push Docker Image, Deploy to Swarm, Verify និង Notification។ Pipeline ក៏ត្រូវបានរៀបចំឱ្យអាច

ជ្រើសរើសសកម្មភាពតាម Parameters ដើម្បីគ្រប់គ្រងការបង្កើត ការបញ្ជូន Image ការដាក់ឱ្យដំណើរការ និងការត្រឡប់ទៅកំណែកម្មវិធីមុន។

តាមលទ្ធផល Jenkins Console Output បានបង្ហាញថាសកម្មភាព Build, Push Image និង Deploy សម្រាប់កំណែកម្មវិធី `v1.0.0` បានបញ្ចប់ដោយស្ថានភាព `Finished: SUCCESS`។ ការបង្ហាញ Pipeline Stages នៅក្នុង Jenkins ក៏ជួយឱ្យក្រុមអាចពិនិត្យមើលលំដាប់ការងារ និងស្ថានភាពនៃដំណាក់កាលនីមួយៗបានផងដែរ។

លទ្ធផលទាំងនេះបង្ហាញថា Jenkins អាចគ្រប់គ្រងដំណើរការ CI/CD ដែលបានកំណត់ និងផ្តល់ព័ត៌មានសម្រាប់តាមដានលទ្ធផលនៃការអនុវត្ត។ កស្តុតាងត្រូវបានបង្ហាញនៅក្នុងរូបភាពទី៥.១១ ដល់ទី៥.១៣ និង Jenkins Pipeline Screenshots ក្នុងផ្នែកលទ្ធផល។

៦.៥. លទ្ធផលនៃការគ្រប់គ្រងកំណែកម្មវិធី និង Rollback

ការគ្រប់គ្រងកំណែកម្មវិធីត្រូវបានរៀបចំតាមរយៈ Git Tags, Docker Image Versions និង Jenkins Pipeline ដើម្បីគាំទ្រការធ្វើបច្ចុប្បន្នភាពកម្មវិធី និងការត្រឡប់ទៅកំណែមុននៅពេលចាំបាច់។

ក្នុងការអនុវត្តគម្រោង ក្រុមបានប្រើប្រាស់កំណែកម្មវិធីផ្សេងៗ ដូចជា `v1.0.0`, `v1.0.1` និង `v1.1.0` ដើម្បីសម្គាល់ការផ្លាស់ប្តូររបស់កម្មវិធី។ Jenkins Pipeline ត្រូវបានរៀបចំឱ្យមានសកម្មភាព Deploy និង Rollback សម្រាប់គ្រប់គ្រងការផ្លាស់ប្តូរកំណែទាំងនេះ។

តាមលទ្ធផល Jenkins Console Output សកម្មភាព Rollback ទៅកំណែ `v1.0.0` បានបញ្ចប់ដោយស្ថានភាព `Finished: SUCCESS`។ លទ្ធផលនេះបញ្ជាក់ថាដំណើរការ Pipeline សម្រាប់ Rollback បានបញ្ចប់ដោយជោគជ័យ។ ទោះជាយ៉ាងណា ការវាយតម្លៃភាពជោគជ័យរបស់ប្រព័ន្ធក្រោយ Rollback ត្រូវការការផ្ទៀងផ្ទាត់បន្ថែមលើស្ថានភាព Services, Replicas និងការចូលប្រើកម្មវិធី។

កស្តុតាងនៃការអនុវត្ត Rollback ត្រូវបានបង្ហាញនៅក្នុងរូបភាពទី៥.១៥។

៦.៦. លទ្ធផលនៃការជូនដំណឹងតាម Telegram

ការជូនដំណឹងតាម Telegram ត្រូវបានរៀបចំឡើង ដើម្បីគាំទ្រដល់ការតាមដានស្ថានភាព Jenkins CI/CD Pipeline និងផ្តល់ព័ត៌មានទៅកាន់សមាជិកក្រុមបន្ទាប់ពីដំណើរការ Build, Deploy ឬ Rollback បានបញ្ចប់។

ក្នុងគម្រោងនេះ Telegram Bot ត្រូវបានភ្ជាប់ជាមួយ Jenkins ដើម្បីផ្ញើសារជូនដំណឹងទៅកាន់ Telegram Group។ លទ្ធផលដែលបានបង្ហាញក្នុងរូបភាពមានសារជូនដំណឹងអំពីការដាក់ឱ្យដំណើរការកម្មវិធីដោយជោគជ័យ ដែលរួមមានព័ត៌មានសំខាន់ៗដូចជា៖

- **Project:** Cloud Computing Capstone Project
- **GitHub Repository:** https://github.com/Chamroeun12/Assignment.git
- **Action:** deploy
- **Version:** Build Number + Short Git Commit SHA
- **Docker Image:** 399311815210.dkr.ecr.ap-southeast-2.amazonaws.com/cicd-assignment
- **Docker Stack:** cicd-assignment
- **Jenkins Job:** [Actual Jenkins Job Name]
- **Build Number:** [Actual Build Number]
- **Status:** Deployment Successful *(if confirmed by Jenkins)*
- **AWS Region:** ap-southeast-2 (Sydney)

- **Deployment Platform:** AWS EC2 + Docker Swarm
- **Container Registry:** Amazon ECR

បន្ថែមពីនេះ Jenkins Console Output បានបង្ហាញសារ `Telegram notification sent successfully.` និងស្ថានភាព `Finished: SUCCESS` ដែលបញ្ជាក់ថា Pipeline បានរាយការណ៍ពីការផ្ញើសារជូនដំណឹងដោយជោគជ័យ។

លទ្ធផលនេះបង្ហាញថាការភ្ជាប់ Jenkins ជាមួយ Telegram Bot អាចប្រើប្រាស់សម្រាប់ជូនដំណឹងអំពីសកម្មភាព Deployment និងលទ្ធផល Pipeline បាន។ វាជួយឱ្យសមាជិកក្រុមទទួលបានព័ត៌មានទាន់ពេលវេលា និងងាយស្រួលតាមដានស្ថានភាពការងារ។

ភស្តុតាងនៃការជូនដំណឹងតាម Telegram ត្រូវបានបង្ហាញនៅក្នុងរូបភាពទី៥.១៦។

៦.៧. ការវាយតម្លៃលទ្ធផលសរុប

ផ្អែកលើភស្តុតាងដែលបានកត់ត្រា គម្រោង Cloud Infrastructure & DevOps Automation បានបង្ហាញពីលទ្ធផលនៃការរួមបញ្ចូលបច្ចេកវិទ្យាសំខាន់ៗក្នុងការរៀបចំ និងគ្រប់គ្រងកម្មវិធីនៅលើ Cloud។ GitHub ត្រូវបានប្រើប្រាស់សម្រាប់គ្រប់គ្រងប្រភពកូដ Docker សម្រាប់វេចខ្ចប់កម្មវិធី Terraform សម្រាប់បង្កើតហេដ្ឋារចនាសម្ព័ន្ធ AWS EC2 និង Docker Swarm សម្រាប់គ្រប់គ្រង Services នៅលើម៉ាស៊ីនមេច្រើន។

ជាមួយគ្នានេះ Nginx បានគាំទ្រការចូលប្រើកម្មវិធីតាមរយៈ HTTP ខណៈ Jenkins ត្រូវបានប្រើសម្រាប់គ្រប់គ្រងលំហូរការងារ CI/CD និងការផ្លាស់ប្តូរកំណែកម្មវិធី។ ការភ្ជាប់ Telegram Bot ក៏បានបង្ហាញលទ្ធផលនៃការផ្ញើសារជូនដំណឹងអំពីស្ថានភាព Deployment ទៅកាន់ក្រុមផងដែរ។

លទ្ធផលសំខាន់ៗដែលបានផ្ទៀងផ្ទាត់រួមមាន ការបង្កើត និងបញ្ជូន Docker Image ទៅ Docker Hub ការដំណើរការ Containers នៅលើ Local ការបង្កើត EC2 Instances ចំនួនបី ការរៀបចំ Docker Swarm Cluster ដែល Nodes មានស្ថានភាព Ready ការដាក់ឱ្យដំណើរការ Services តាមចំនួន Replicas ដែលបានកំណត់ និងការចូលប្រើកម្មវិធីតាមរយៈ Public IP Address។ លើសពីនេះ Jenkins Pipeline បានបង្ហាញលទ្ធផលជោគជ័យសម្រាប់សកម្មភាព Build, Push, Deploy និង Rollback ព្រមទាំងការជូនដំណឹងតាម Telegram។

សរុបមក លទ្ធផលនៃគម្រោងបានបង្ហាញពីការអនុវត្តជាក់ស្តែងនៃការរួមបញ្ចូល Cloud Computing និង DevOps ដើម្បីគាំទ្រដល់ការរៀបចំហេដ្ឋារចនាសម្ព័ន្ធ ការដាក់ឱ្យដំណើរការកម្មវិធី និងការគ្រប់គ្រងដំណើរការការងារ។ ទោះបីជាយ៉ាងណា ការវាយតម្លៃលើភាពធន់នឹងការបរាជ័យរបស់ប្រព័ន្ធ សុវត្ថិភាព និងសមត្ថភាពស្តារសេវាកម្មក្រោយមានបញ្ហា នៅតែត្រូវការការធ្វើតេស្ត និងការផ្ទៀងផ្ទាត់បន្ថែម ដើម្បីធានាថាប្រព័ន្ធអាចដំណើរការបានប្រកបដោយភាពជឿជាក់ក្នុងការប្រើប្រាស់ជាក់ស្តែង។

៧. ការពិភាក្សា

ផ្អែកលើដំណើរការអនុវត្ត និងលទ្ធផលដែលបានបង្ហាញក្នុងផ្នែកទី៥ និងទី៦ គម្រោង Cloud Infrastructure & DevOps Automation បានបង្ហាញពីការរួមបញ្ចូលបច្ចេកវិទ្យា Cloud Computing និង DevOps ដើម្បីគាំទ្រដល់ការអភិវឌ្ឍ ការគ្រប់គ្រង និងការដាក់ឱ្យដំណើរការកម្មវិធីនៅលើ Cloud។ ការអនុវត្តគម្រោងនេះមិនត្រឹមតែផ្តោតលើការបង្កើតហេដ្ឋារចនាសម្ព័ន្ធ និងការដាក់ឱ្យដំណើរការកម្មវិធីប៉ុណ្ណោះទេ ប៉ុន្តែថែមទាំងបង្ហាញពីសារៈសំខាន់នៃការប្រើ

ប្រាស់ឧបករណ៍ស្វ័យប្រវត្តិកម្ម ការគ្រប់គ្រងកំណែកម្មវិធី និងការផ្ទៀងផ្ទាត់លទ្ធផលនៅក្នុងដំណើរការ Deployment ផងដែរ។

ការពិភាក្សានេះផ្តោតលើការវិភាគចំណុចខ្លាំង និងចំណុចខ្សោយនៃប្រព័ន្ធដែលបានរៀបចំ ដោយពិចារណាលើប្រសិទ្ធភាពនៃការរួមបញ្ចូលបច្ចេកវិទ្យា ភាពងាយស្រួលក្នុងការគ្រប់គ្រង ស្វ័យប្រវត្តិកម្ម សុវត្ថិភាព និងសមត្ថភាពក្នុងការស្តារប្រព័ន្ធនៅពេលមានបញ្ហា។ ការវិភាគនេះក៏ជួយកំណត់ដែនកំណត់នៃការអនុវត្ត និងផ្តល់ជាទិសដៅសម្រាប់ការកែលម្អប្រព័ន្ធឱ្យកាន់តែមានប្រសិទ្ធភាពនាពេលអនាគត។

៧.១. ចំណុចខ្លាំង

តាមរយៈលទ្ធផលនៃការអនុវត្ត គម្រោងមានចំណុចខ្លាំងសំខាន់ៗមួយចំនួនដូចខាងក្រោម៖

១. ការរួមបញ្ចូលបច្ចេកវិទ្យាក្នុងលំហូរការងារតែមួយ

គម្រោងបានបង្ហាញពីការរួមបញ្ចូលបច្ចេកវិទ្យាសំខាន់ៗ ដូចជា GitHub, Jenkins, Terraform, Docker, Docker Swarm, AWS EC2 និង Nginx ក្នុងដំណើរការអភិវឌ្ឍ និងដាក់ឱ្យដំណើរការកម្មវិធី។ បច្ចេកវិទ្យានីមួយៗមានតួនាទីជាក់លាក់ និងគាំទ្រគ្នាទៅវិញទៅមក ចាប់ពីការគ្រប់គ្រងប្រភពកូដ ការបង្កើត Docker Image ការរៀបចំហេដ្ឋារចនាសម្ព័ន្ធ Cloud រហូតដល់ការដាក់ឱ្យដំណើរការកម្មវិធី និងការផ្ទៀងផ្ទាត់លទ្ធផល។

ការរួមបញ្ចូលនេះជួយឱ្យដំណើរការការងារមានលំដាប់លំដោយ និងធ្វើឱ្យការគ្រប់គ្រងសមាសភាគផ្សេងៗរបស់ប្រព័ន្ធកាន់តែងាយស្រួល។

២. ការកាត់បន្ថយការងារដោយដៃ និងការអនុវត្តឡើងវិញ

ការប្រើប្រាស់ Terraform តាមគោលការណ៍ Infrastructure as Code (IaC) អនុញ្ញាតឱ្យក្រុមកំណត់ និងបង្កើតហេដ្ឋារចនាសម្ព័ន្ធ AWS តាមរយៈ Configuration Files ជំនួសឱ្យការរៀបចំដោយដៃ។ ចំណែក Dockerfile និង Docker Compose ជួយកំណត់បរិស្ថានដំណើរការរបស់កម្មវិធីឱ្យមានភាពស្របគ្នា ខណៈ Jenkinsfile អនុញ្ញាតឱ្យកំណត់ដំណាក់កាលនៃ CI/CD Pipeline ជា Code។

ការរៀបចំបែបនេះជួយកាត់បន្ថយភាពស្មុគស្មាញក្នុងការកំណត់រចនាសម្ព័ន្ធ និងធ្វើឱ្យការអនុវត្តដំណើរការដដែលៗមានភាពងាយស្រួល។ លើសពីនេះ វាក៏ជួយកាត់បន្ថយហានិភ័យនៃកំហុសដែលអាចកើតឡើងពីការកំណត់ដោយដៃផងដែរ។

៣. ការសាកល្បង និងផ្ទៀងផ្ទាត់កម្មវិធីតាមដំណាក់កាល

គម្រោងបានរៀបចំការសាកល្បងកម្មវិធីចាប់ពី Local Environment ដោយប្រើ Docker Compose មុនពេលបន្តទៅការដាក់ឱ្យដំណើរការលើ AWS Cloud។ ការសាកល្បងដំបូងនេះជួយពិនិត្យមើលការដំណើរការរបស់ Application, Database និង Nginx ព្រមទាំងការតភ្ជាប់រវាងសេវាកម្មទាំងនោះ។

បន្ទាប់ពីដាក់ឱ្យដំណើរការលើ Cloud ការផ្ទៀងផ្ទាត់ត្រូវបានអនុវត្តតាមរយៈការពិនិត្យ EC2 Instances, Docker Swarm Nodes, Services, Replicas និង HTTP Responses។ ការរៀបចំការសាកល្បងតាមដំណាក់កាលបែបនេះជួយឱ្យក្រុមអាចតាមដានលទ្ធផល និងស្វែងរកបញ្ហាដែលអាចកើតមានក្នុងដំណើរការ Deployment បានកាន់តែងាយស្រួល។

៤. ការគ្រប់គ្រងកំណែកម្មវិធី និងដំណើរការ Deployment

ការប្រើប្រាស់ Git, GitHub និង Docker Image Tags ជួយឱ្យគម្រោងអាចសម្គាល់ និងតាមដានកំណែកម្មវិធីបានច្បាស់លាស់។ ជាមួយគ្នានេះ Jenkins CI/CD Pipeline ត្រូវបានរៀបចំសម្រាប់គ្រប់គ្រងសកម្មភាព Build, Push Image, Deploy និង Rollback ដែលជួយឱ្យការផ្លាស់ប្តូរកំណែកម្មវិធីមានលំដាប់លំដោយ។

ការប្រើប្រាស់ Docker Swarm ក៏អនុញ្ញាតឱ្យគ្រប់គ្រងចំនួន Services និង Replicas នៅលើម៉ាស៊ីនមេច្រើន ព្រមទាំងគាំទ្រការធ្វើ Rolling Update។ លទ្ធផល Jenkins Pipeline ដែលបានបង្ហាញស្ថានភាពជោគជ័យសម្រាប់សកម្មភាព Deploy និង Rollback គឺជាភស្តុតាងមួយនៃការអនុវត្តដំណើរការគ្រប់គ្រងកំណែកម្មវិធី។

៥. ការយកចិត្តទុកដាក់លើសុវត្ថិភាព និងការជូនដំណឹង

គម្រោងបានរួមបញ្ចូលគោលការណ៍សុវត្ថិភាពក្នុងការគ្រប់គ្រងព័ត៌មានសម្ងាត់ និងការកំណត់សិទ្ធិចូលប្រើប្រព័ន្ធ។ Jenkins Credentials ត្រូវបានកំណត់ជាមធ្យោបាយសម្រាប់គ្រប់គ្រងព័ត៌មានសម្ងាត់ដែលត្រូវប្រើប្រាស់ក្នុង CI/CD Pipeline ខណៈ AWS Security Groups ត្រូវបានប្រើសម្រាប់កំណត់ចរាចរណ៍បណ្តាញតាមតម្រូវការរបស់គម្រោង។

លើសពីនេះ ការភ្ជាប់ Jenkins ជាមួយ Telegram Bot បានជួយគាំទ្រដល់ការជូនដំណឹងអំពីស្ថានភាព Pipeline។ លទ្ធផលដែលបានបង្ហាញក្នុងរបាយការណ៍បញ្ជាក់ថាការជូនដំណឹងអំពីសកម្មភាព Deployment អាចត្រូវបានផ្ញើទៅកាន់ Telegram Group ដើម្បីឱ្យសមាជិកក្រុមទទួលបានព័ត៌មានអំពីលទ្ធផលការងារបានឆាប់រហ័ស។

៧.២. ចំណុចខ្សោយ និងដែនកំណត់

ទោះបីជាគម្រោងបានបង្ហាញលទ្ធផលវិជ្ជមានក្នុងការរួមបញ្ចូលបច្ចេកវិទ្យា និងការដាក់ឱ្យដំណើរការកម្មវិធីក៏ដោយ ប្រព័ន្ធនៅតែមានដែនកំណត់មួយចំនួនដែលគួរត្រូវបានពិចារណា និងកែលម្អបន្ថែម។

១. ដែនកំណត់នៃភាពធន់នឹងការបរាជ័យរបស់ Docker Swarm Manager

ក្នុងការរៀបចំ Docker Swarm Cluster គម្រោងបានប្រើប្រាស់ Manager Node ចំនួនមួយ និង Worker Nodes ចំនួនពីរ។ រចនាសម្ព័ន្ធនេះអាចគាំទ្រការគ្រប់គ្រង និងដំណើរការ Services នៅលើម៉ាស៊ីនមេច្រើន ប៉ុន្តែការមាន Manager Node តែមួយអាចបង្កើតជាចំណុចខ្សោយនៃប្រព័ន្ធ (Single Point of Failure) សម្រាប់ការគ្រប់គ្រង Cluster។

ក្នុងករណី Manager Node ឈប់ដំណើរការ Containers ដែលកំពុងដំណើរការលើ Worker Nodes អាចបន្តបម្រើសេវាកម្មបាន ប៉ុន្តែការគ្រប់គ្រង Cluster និងការធ្វើបច្ចុប្បន្នភាព Services អាចត្រូវបានរំខានរហូតដល់ Manager ត្រូវបានស្តារឡើងវិញ។

ដើម្បីកែលម្អបញ្ហានេះ នាពេលអនាគតគម្រោងអាចពិចារណាប្រើប្រាស់ Manager Nodes ច្រើន និងរៀបចំរចនាសម្ព័ន្ធ High Availability ដើម្បីបង្កើនភាពធន់របស់ប្រព័ន្ធ។

២. ដែនកំណត់នៃការគ្រប់គ្រងសិទ្ធិ និងការអនុម័តក្នុង Jenkins

ទោះបីជា Jenkins CI/CD Pipeline បានគាំទ្រសកម្មភាព Build, Deploy និង Rollback ក៏ដោយ ការកំណត់សិទ្ធិ និងការអនុម័តមុនពេលអនុវត្តសកម្មភាពសំខាន់ៗនៅតែជាចំណុចដែលគួរពង្រឹងបន្ថែម។

ប្រសិនបើ Pipeline អនុញ្ញាតឱ្យអ្នកប្រើប្រាស់អនុវត្តសកម្មភាព Deploy ឬ Rollback ដោយគ្មានការត្រួតពិនិត្យសិទ្ធិ និងការអនុម័តសមស្រប នោះអាចបង្កើនហានិភ័យនៃការផ្លាស់ប្តូរប្រព័ន្ធដោយមិនបានគ្រោងទុក។

ដូច្នេះ ការកំណត់ Role-Based Access Control (RBAC) និងការបន្ថែមដំណាក់កាលអនុម័ត (Approval Stage) សម្រាប់សកម្មភាពសំខាន់ៗ គឺជាវិធីសាស្ត្រដែលអាចជួយពង្រឹងការគ្រប់គ្រង និងសុវត្ថិភាពនៃ CI/CD Pipeline។

៣. ការធ្វើតេស្តស្វ័យប្រវត្តិ និងការត្រួតពិនិត្យប្រព័ន្ធនៅមានកម្រិត

គម្រោងបានបង្ហាញការសាកល្បងកម្មវិធី និងការធ្វើងផ្ទាត់លទ្ធផលតាមរយៈ Docker Compose, Jenkins Pipeline, HTTP Response និង Docker Swarm Services។ ទោះជាយ៉ាងណា ភស្តុតាងដែលបានបង្ហាញមិនទាន់គ្រប់គ្រាន់សម្រាប់វាយតម្លៃលើការធ្វើតេស្តស្វ័យប្រវត្តិគ្រប់ប្រភេទ ដូចជា Unit Testing, Integration Testing, Performance Testing និង Load Testing នៅឡើយទេ។

ជាមួយគ្នានេះ ការត្រួតពិនិត្យស្ថានភាពប្រព័ន្ធជាបន្តបន្ទាប់ ដូចជា CPU Usage, Memory Usage, Network Traffic និង Application Errors មិនទាន់មានភស្តុតាងលម្អិតគ្រប់គ្រាន់ក្នុងរបាយការណ៍។ ការបន្ថែមប្រព័ន្ធ Monitoring និងការធ្វើតេស្តទាំងនេះអាចជួយឱ្យការវាយតម្លៃសមត្ថភាព និងស្ថិរភាពរបស់ប្រព័ន្ធមានភាពច្បាស់លាស់ជាងមុន។

៤. ការធ្វើងផ្ទាត់ក្រោយ Rolling Update និង Rollback

លទ្ធផល Jenkins Console Output បានបង្ហាញថាសកម្មភាព Deploy និង Rollback អាចបញ្ចប់ដោយស្ថានភាពជោគជ័យ។ ទោះជាយ៉ាងណា ស្ថានភាព SUCCESS របស់ Pipeline មិនមានន័យថាកម្មវិធីទាំងមូលបានដំណើរការត្រឹមត្រូវក្នុងគ្រប់លក្ខខណ្ឌនោះទេ។

ដើម្បីធានាប្រសិទ្ធភាពនៃការផ្លាស់ប្តូរកំណែកម្មវិធី គម្រោងគួរពង្រឹងការធ្វើងផ្ទាត់ក្រោយ Deployment និង Rollback ដូចជា ការពិនិត្យ Service Health, Replicas, HTTP Response និង Application Logs។ ការអនុវត្តនេះនឹងជួយឱ្យក្រុមអាចកំណត់បញ្ហាបានលឿន និងមានភស្តុតាងច្បាស់លាស់សម្រាប់វាយតម្លៃលទ្ធផលនៃការស្តារប្រព័ន្ធ។

៧.៣. ទិសដៅកែលម្អនាពេលអនាគត

ដើម្បីបង្កើនប្រសិទ្ធភាព និងភាពជឿជាក់នៃប្រព័ន្ធ គម្រោងអាចត្រូវបានកែលម្អបន្ថែមតាមរយៈការពង្រឹងរចនាសម្ព័ន្ធ Docker Swarm ឱ្យមាន High Availability ការកំណត់សិទ្ធិ និងការអនុម័តក្នុង Jenkins CI/CD Pipeline ការពង្រីកការធ្វើតេស្តស្វ័យប្រវត្តិ និងការរៀបចំប្រព័ន្ធត្រួតពិនិត្យស្ថានភាពសេវាកម្មជាបន្តបន្ទាប់។

ជាពិសេស ការប្រើប្រាស់ឧបករណ៍ Monitoring និង Logging អាចជួយក្រុមតាមដានស្ថានភាព Infrastructure និង Application បានកាន់តែច្បាស់លាស់។ ការបន្ថែម Health Checks និងការធ្វើងផ្ទាត់លទ្ធផលក្រោយ Rolling Update ឬ Rollback ក៏អាចជួយកាត់បន្ថយហានិភ័យនៃការរំខានដល់សេវាកម្ម និងធ្វើឱ្យការគ្រប់គ្រងប្រព័ន្ធមានភាពងាយស្រួលជាងមុន។

សរុបមក គម្រោងមានចំណុចខ្លាំងលើការរួមបញ្ចូលបច្ចេកវិទ្យា និងការរៀបចំលំហូរការងារ DevOps ដោយបានបង្ហាញលទ្ធផលជាក់ស្តែងក្នុងការបង្កើតហេដ្ឋារចនាសម្ព័ន្ធ និងការដាក់ឱ្យដំណើរការកម្មវិធី។ ចំណែកដែនកំណត់ដែលបានពិភាក្សាខាងលើ គឺជាចំណុចដែលត្រូវយកចិត្តទុកដាក់ក្នុងការអភិវឌ្ឍបន្ថែម មិនមែនជាការបញ្ជាក់ថាប្រព័ន្ធបានបរាជ័យក្នុងការអនុវត្តនោះទេ។

៨. សេចក្តីសន្និដ្ឋាន

តាមរយៈការសិក្សា និងអនុវត្តគម្រោង Cloud Infrastructure & DevOps Automation បានបង្ហាញពីសារៈសំខាន់នៃការរួមបញ្ចូលបច្ចេកវិទ្យា Cloud Computing និងគោលការណ៍ DevOps ក្នុងការរៀបចំហេដ្ឋារចនាសម្ព័ន្ធ ការអភិវឌ្ឍ និងការដាក់ឱ្យដំណើរការកម្មវិធី។ គម្រោងនេះបានផ្តោតលើការប្រើប្រាស់បច្ចេកវិទ្យា និងឧបករណ៍សំខាន់ៗ រួមមាន GitHub, Jenkins, Terraform, Docker, Docker Swarm, AWS EC2 និង Nginx ដើម្បីបង្កើតលំហូរការងារដែលមានលំដាប់លំដោយ និងអាចគ្រប់គ្រងបានកាន់តែងាយស្រួល។

ក្នុងដំណើរការអនុវត្ត GitHub ត្រូវបានប្រើប្រាស់សម្រាប់គ្រប់គ្រងប្រភពកូដ និងតាមដានការផ្លាស់ប្តូរកម្មវិធី ខណៈ Docker ត្រូវបានប្រើសម្រាប់វេចខ្ចប់កម្មវិធីទៅជា Container Image ដើម្បីគាំទ្រការដំណើរការនៅលើបរិស្ថានផ្សេងៗ។ Terraform ត្រូវបានប្រើប្រាស់សម្រាប់កំណត់ និងបង្កើតហេដ្ឋារចនាសម្ព័ន្ធនៅលើ AWS តាមគោលការណ៍ Infrastructure as Code (IaC)។ ចំណែក Docker Swarm ត្រូវបានប្រើសម្រាប់រៀបចំ Cluster និងគ្រប់គ្រង Services នៅលើ EC2 Instances ហើយ Nginx ត្រូវបានប្រើសម្រាប់គាំទ្រការចូលប្រើកម្មវិធីតាមរយៈ HTTP។

លើសពីនេះ Jenkins ត្រូវបានប្រើប្រាស់សម្រាប់រៀបចំ CI/CD Pipeline ដើម្បីគាំទ្រដល់ដំណើរការ Build, Test, Push Image, Deploy និង Verify។ ការភ្ជាប់ Jenkins ជាមួយ GitHub តាមរយៈ Webhook បានគាំទ្រដល់ការជូនដំណឹងអំពីការផ្លាស់ប្តូរប្រភពកូដ ខណៈការរៀបចំសកម្មភាព Deploy និង Rollback បានជួយឱ្យការគ្រប់គ្រងកំណែកម្មវិធីមានលំដាប់ច្បាស់លាស់។ ជាមួយគ្នានេះ ការភ្ជាប់ Telegram Bot បានគាំទ្រដល់ការផ្ញើសារជូនដំណឹងអំពីស្ថានភាព CI/CD Pipeline ទៅកាន់សមាជិកក្រុម។

ផ្អែកលើលទ្ធផលដែលបានកត់ត្រា គម្រោងបានបង្ហាញពីការបង្កើត Docker Images ការសាកល្បងកម្មវិធីនៅលើ Local Environment ការបង្កើតហេដ្ឋារចនាសម្ព័ន្ធ AWS EC2 ការរៀបចំ Docker Swarm Cluster និងការដាក់ឱ្យដំណើរការ Services តាមចំនួន Replicas ដែលបានកំណត់។ លទ្ធផល Jenkins Pipeline ក៏បានបង្ហាញពីការបញ្ចប់សកម្មភាព Build, Deploy និង Rollback ដោយជោគជ័យ ព្រមទាំងការជូនដំណឹងតាម Telegram។ ភស្តុតាងទាំងនេះបានបង្ហាញពីការអនុវត្តជាក់ស្តែងនៃលំហូរការងារ Cloud និង DevOps ដែលបានរៀបចំក្នុងគម្រោង។

ទោះជាយ៉ាងណា ការសិក្សានេះក៏បានបង្ហាញថា ការដាក់ឱ្យដំណើរការកម្មវិធីដោយជោគជ័យមិនមែនជាគោលដៅតែមួយនៃការអនុវត្ត DevOps នោះទេ។ ការរៀបចំប្រព័ន្ធឱ្យមានសុវត្ថិភាព ភាពអាចអនុវត្តឡើងវិញបាន សមត្ថភាពតាមដានលទ្ធផល និងភាពធន់នឹងបញ្ហា គឺជាកត្តាសំខាន់ដែលត្រូវយកចិត្តទុកដាក់ផងដែរ។ ដែនកំណត់មួយចំនួន ដូចជា ការប្រើប្រាស់ Docker Swarm Manager តែមួយ ការគ្រប់គ្រងសិទ្ធិក្នុង Jenkins និងការផ្ទៀងផ្ទាត់ស្ថានភាពកម្មវិធីក្រោយ Deployment ឬ Rollback នៅតែត្រូវការការកែលម្អ និងការធ្វើតេស្តបន្ថែម។

ជារួម គម្រោងនេះបានផ្តល់នូវចំណេះដឹង និងបទពិសោធន៍ជាក់ស្តែងក្នុងការរួមបញ្ចូលបច្ចេកវិទ្យា Cloud Computing និង DevOps ដើម្បីរៀបចំ គ្រប់គ្រង និងដាក់ឱ្យដំណើរការកម្មវិធីនៅលើ Cloud។ ចំណេះដឹងដែលទទួលបានអាចយកទៅអនុវត្ត និងអភិវឌ្ឍបន្ថែមក្នុងគម្រោងនាពេលអនាគត ជាពិសេសការបង្កើនស្វ័យប្រវត្តិកម្ម ការពង្រឹងសុវត្ថិភាព ការគ្រប់គ្រងធនធាន និងការកែលម្អសមត្ថភាពតាមដានប្រព័ន្ធ ដើម្បីឱ្យការដាក់ឱ្យដំណើរការកម្មវិធីមានប្រសិទ្ធភាព ស្ថិរភាព និងភាពជឿជាក់កាន់តែប្រសើរឡើង ៕